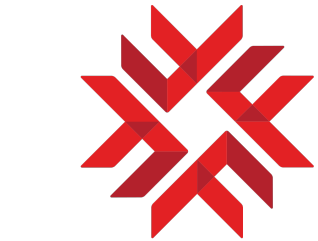
Natural Language Processing 2

Building an LLM from Scratch—Instruction Finetuning

Fatemeh Dehrouyeh



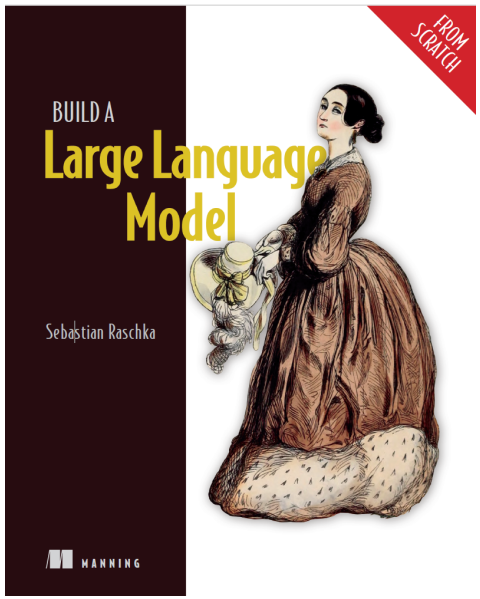
FANSHAWE

Contents

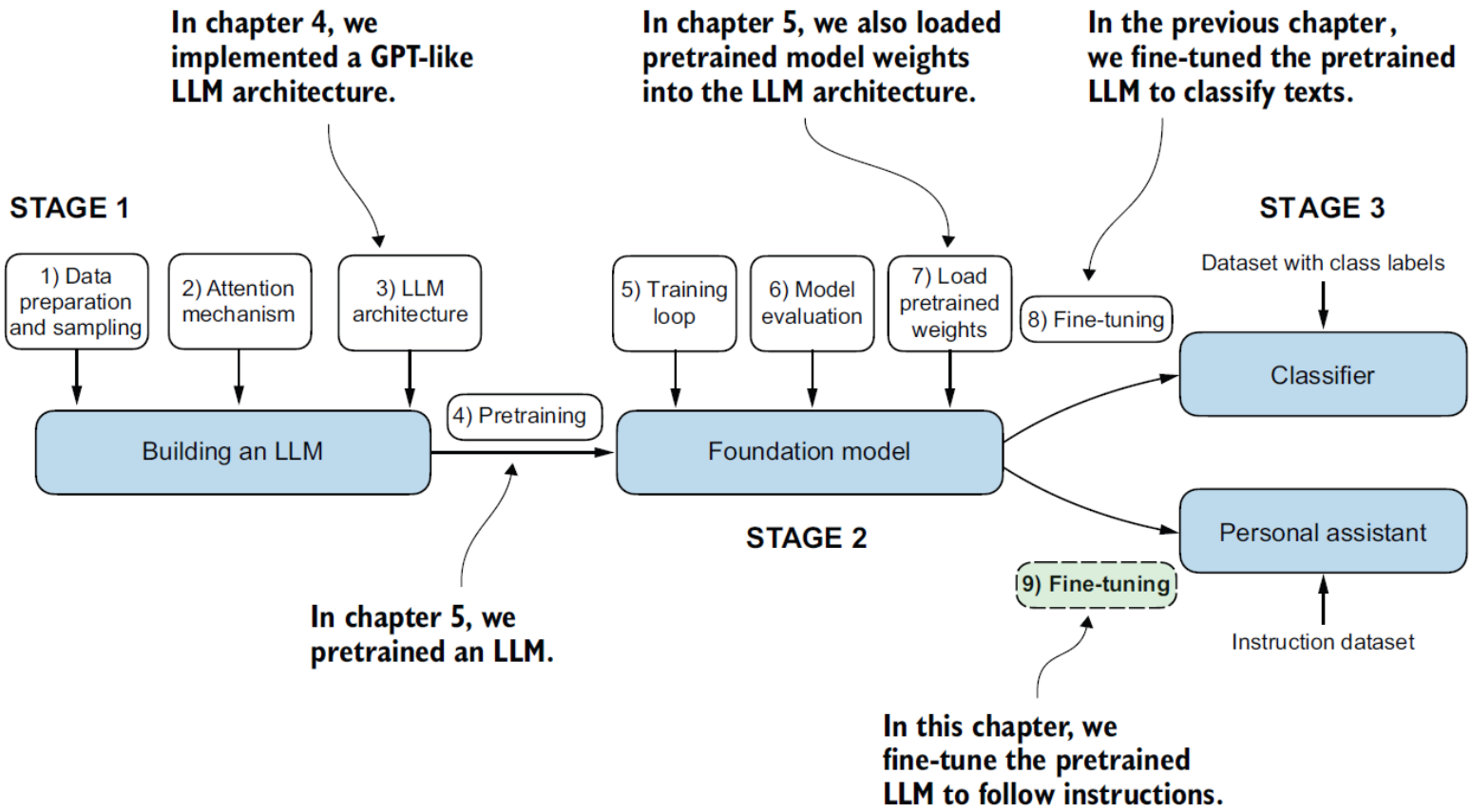
1. Preparing a dataset for supervised instruction finetuning
2. Organizing data into training batches
3. Creating data loaders for an instruction dataset
4. Loading a pretrained LLM
5. Finetuning the LLM on instruction data
6. Extracting and saving responses
7. Evaluating the finetuned LLM

Main Source

- **Video lectures:** YouTube lecture series
- **Source book:** *Build a Large Language Model (From Scratch)* by Sebastian Raschka
- **GitHub code:** `https://github.com/rasbt/LLMs-from-scratch`

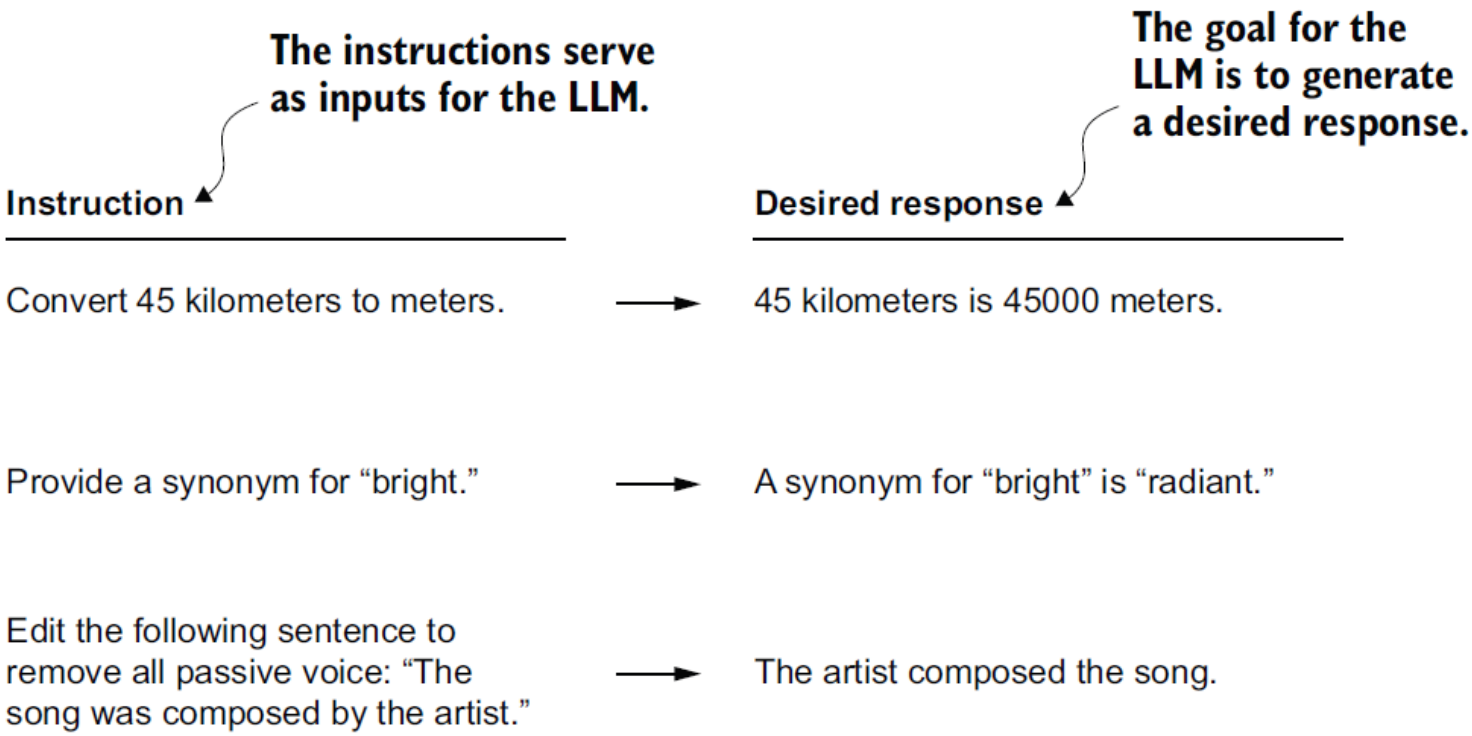


Roadmap


In chapter 4, we implemented a GPT-like LLM architecture.
In chapter 5, we also loaded pretrained model weights into the LLM architecture.
In the previous chapter, we fine-tuned the pretrained LLM to classify texts.
STAGE 1
1) Data preparation and sampling
2) Attention mechanism
3) LLM architecture
Building an LLM
4) Pretraining
In chapter 5, we pretrained an LLM.
5) Training loop
6) Model evaluation
7) Load pretrained weights
Foundation model
STAGE 2
8) Fine-tuning
9) Fine-tuning
In this chapter, we fine-tune the pretrained LLM to follow instructions.
STAGE 3
Dataset with class labels
Classifier
Personal assistant
Instruction dataset

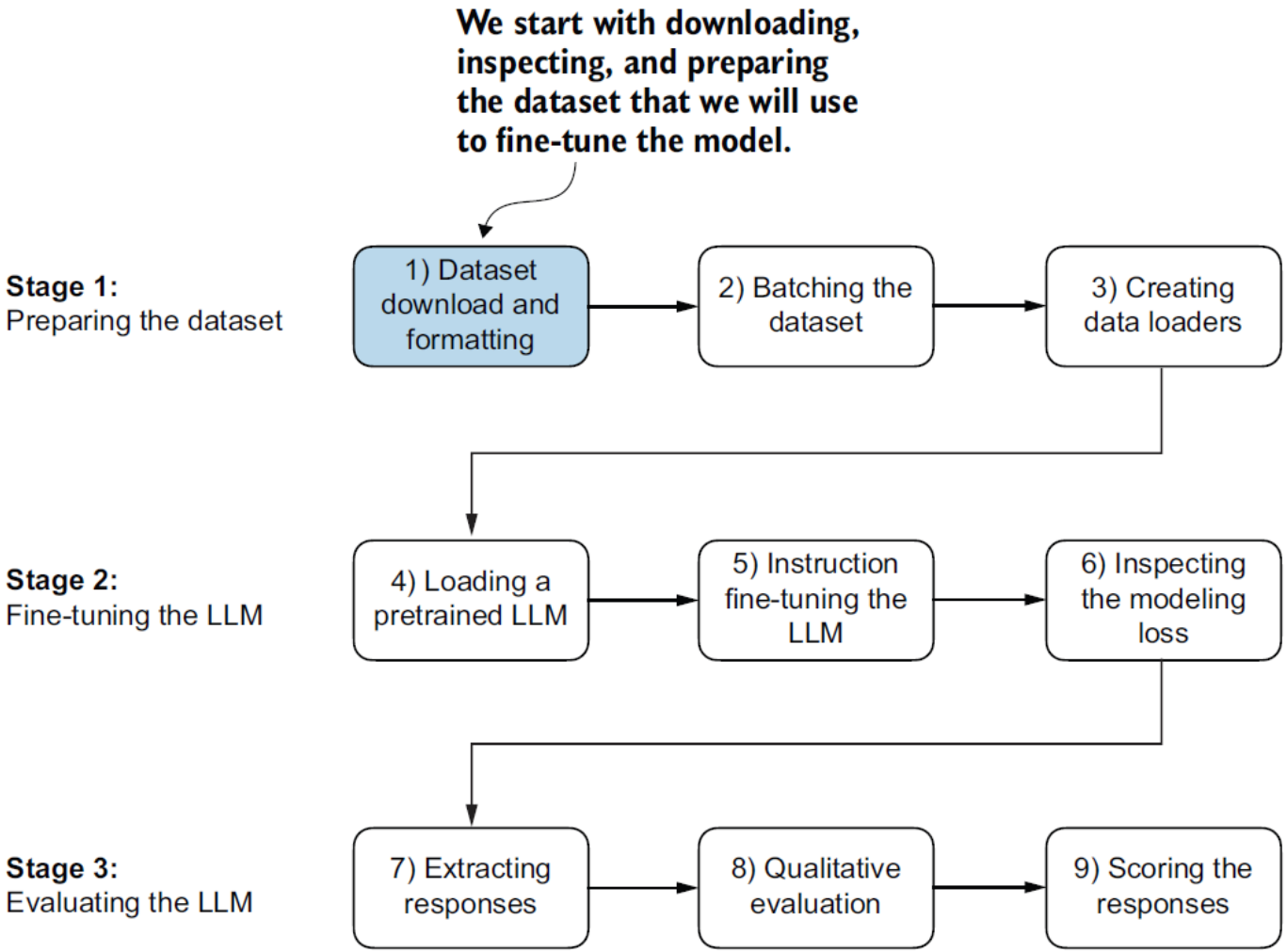
Introduction to instruction finetuning

We saw that pretraining an LLM involves a training procedure where it learns to generate one word at a time. Hence, a pretrained LLM is good at text completion, but it is not good at following instructions. In this chapter, we teach the LLM to follow instructions better.



Topics

The topics covered in this chapter are summarized in the figure below:



Current Section

1 Preparing a dataset for supervised instruction finetuning

2 Organizing data into training batches

3 Creating data loaders for an instruction dataset

4 Loading a pretrained LLM

5 Finetuning the LLM on instruction data

6 Extracting and saving responses

7 Evaluating the finetuned LLM

Preparing a dataset for supervised instruction finetuning

We will work with an instruction dataset prepared for this chapter:

```python
import json
import os
import urllib


def download_and_load_file(file_path, url):

    if not os.path.exists(file_path):
        with urllib.request.urlopen(url) as response:
            text_data = response.read().decode("utf-8")
        with open(file_path, "w", encoding="utf-8") as file:
            file.write(text_data)

    with open(file_path, "r", encoding="utf-8") as file:
        data = json.load(file)

    return data


file_path = "instruction-data.json"
url = (
    "https://raw.githubusercontent.com/rasbt/LLMs-from-scratch"
    "/main/ch07/01_main-chapter-code/instruction-data.json"
)

data = download_and_load_file(file_path, url)
print("Number of entries:", len(data))
```

```
Number of entries: 1100
```

Examples

Each item in the data list we loaded from the JSON file above is a dictionary in the following form:

```
print("Example entry:\n", data[50])
```

```
Example entry:
 {'instruction': 'Identify the correct spelling of the following word.', 'input': 'Ocassion', 'output': "The correct spelling is 'Occasion.'"}
```

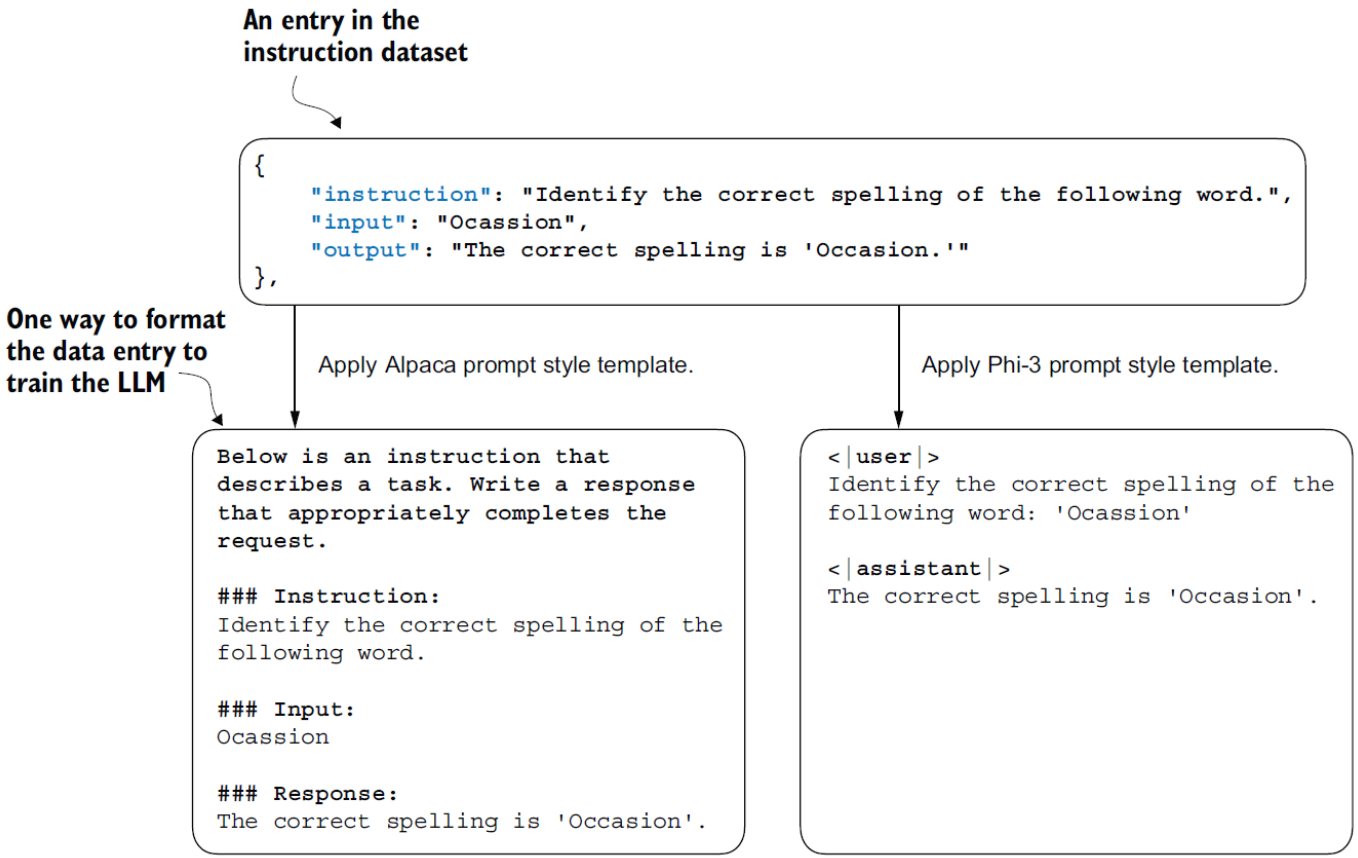
Note that the "input" field can be empty:

```
print("Another example entry:\n", data[999])
```

```
Another example entry:
 {'instruction': "What is an antonym of 'complicated'?", 'input': '', 'output': "An antonym of 'complicated' is 'simple'."}
```

Instruction finetuning

Instruction finetuning is often referred to as "supervised instruction finetuning" because it involves training a model on a dataset where the input-output pairs are explicitly provided. There are different ways to format the entries as inputs to the LLM; the figure below illustrates two example formats that were used for training: the Alpaca (https://crfm.stanford.edu/2023/03/13/alpaca.html) and Phi-3 (https://arxiv.org/abs/2404.14219) LLMs, respectively:



Format-Input Function

In this chapter, we use Alpaca-style prompt formatting, which was the original prompt template for instruction finetuning. Below, we format the input that we will pass as input to the LLM:

```
def format_input(entry):
    instruction_text = (
        f"Below is an instruction that describes a task. "
        f"Write a response that appropriately completes the request."
        f"\n\n### Instruction:\n{entry['instruction']}"
    )

    input_text = f"\n\n### Input:\n{entry['input']}" if entry["input"] else ""

    return instruction_text + input_text
```

Formatted Examples

Formatted responses with an input field and without an input field are shown as below:

```
model_input = format_input(data[50])
desired_response = f"\n\n### Response:\n{data[50]['output']}"

print(model_input + desired_response)
```

```
Below is an instruction that describes a task. Write a response that appropriately completes the request.

### Instruction:
Identify the correct spelling of the following word.

### Input:
Ocassion

### Response:
The correct spelling is 'Occasion.'
```

```
model_input = format_input(data[999])
desired_response = f"\n\n### Response:\n{data[999]['output']}"

print(model_input + desired_response)
```

```
Below is an instruction that describes a task. Write a response that appropriately completes the request.

### Instruction:
What is an antonym of 'complicated'?

### Response:
An antonym of 'complicated' is 'simple'.
```

Note: We do not input the data into LLMs in a special format because the builders have taken care of it behind the scenes, so we do not see it.

Code

Lastly, before we prepare the PyTorch data loaders in the next section, we divide the dataset into a training, validation, and test set:

```
train_portion = int(len(data) * 0.85)  # 85% for training
test_portion = int(len(data) * 0.1)    # 10% for testing
val_portion = len(data) - train_portion - test_portion  # Remaining 5% for validation

train_data = data[:train_portion]
test_data = data[train_portion:train_portion + test_portion]
val_data = data[train_portion + test_portion:]
```

```
print("Training set length:", len(train_data))
print("Validation set length:", len(val_data))
print("Test set length:", len(test_data))
```

```
Training set length: 935
Validation set length: 55
Test set length: 110
```

Current Section

1 Preparing a dataset for supervised instruction finetuning

2 Organizing data into training batches

3 Creating data loaders for an instruction dataset

4 Loading a pretrained LLM

5 Finetuning the LLM on instruction data

6 Extracting and saving responses

7 Evaluating the finetuned LLM

Organizing data into training batches

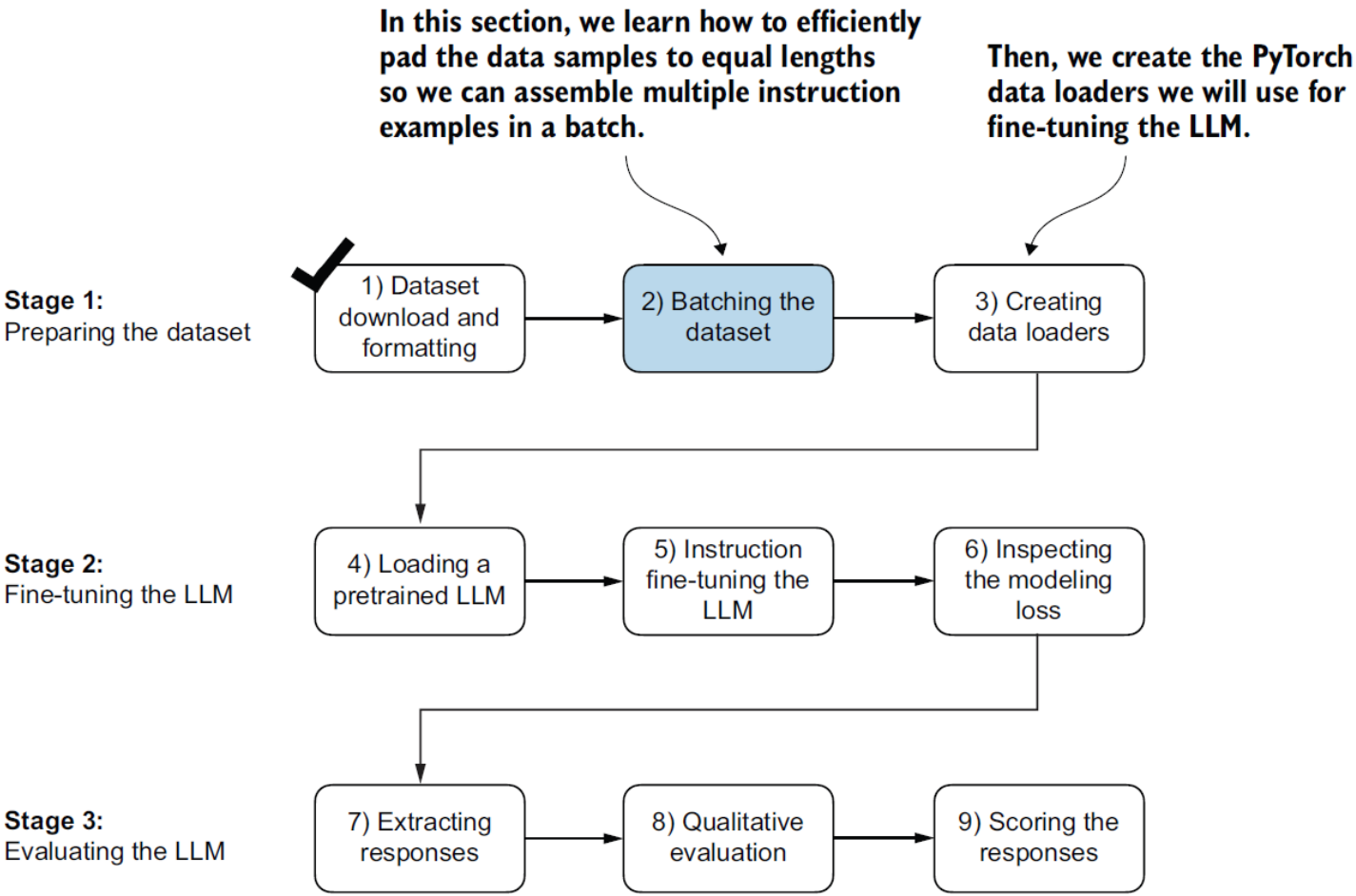
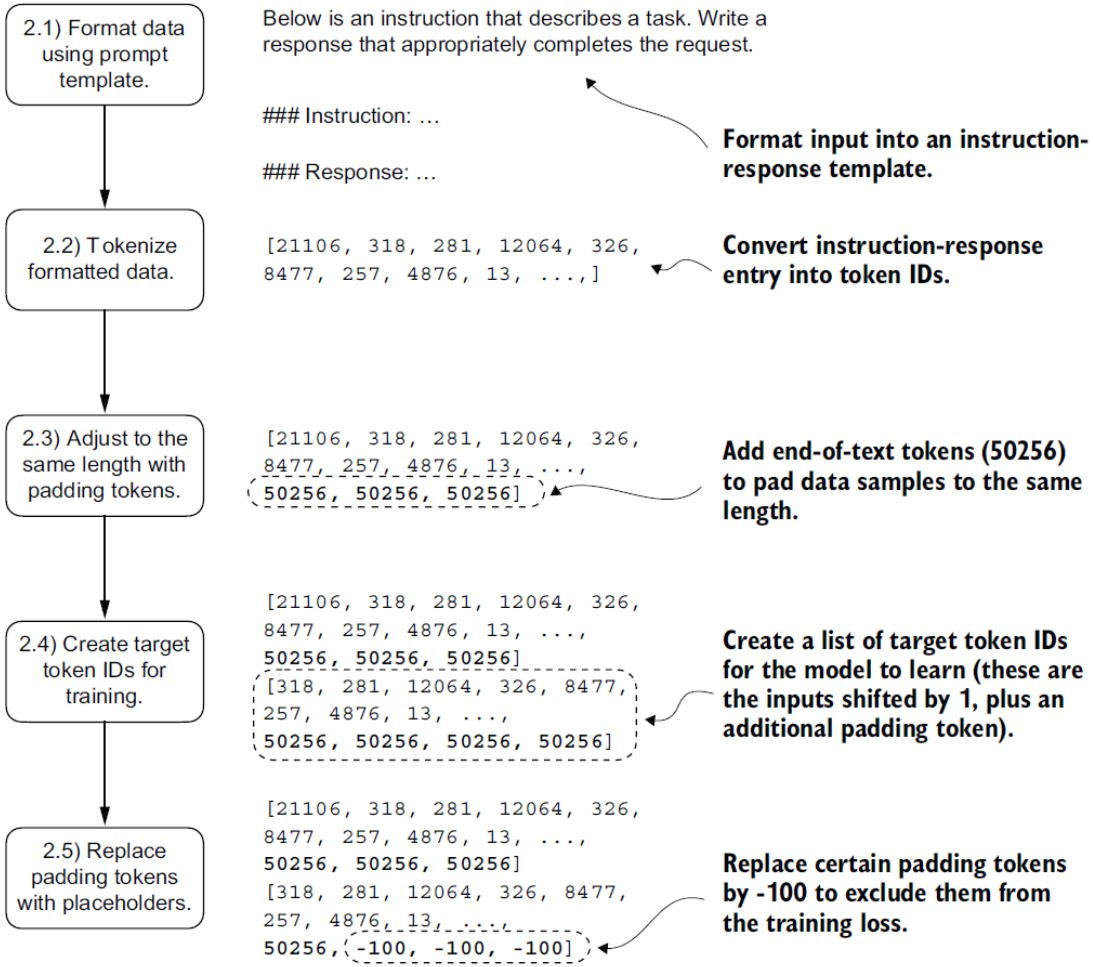


Illustration of Batching the Dataset

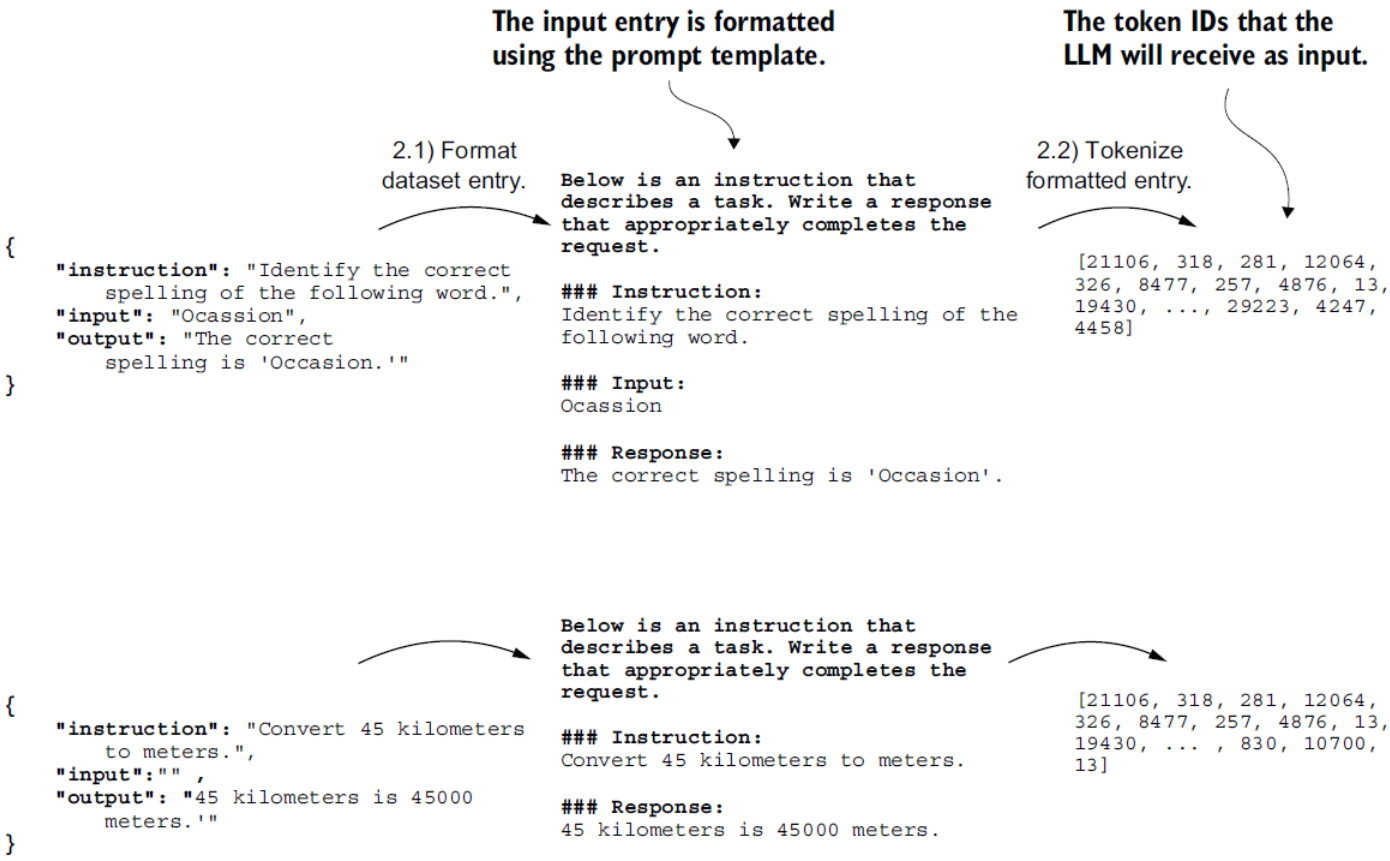
We tackle this dataset batching in several steps, as summarized in the figure below:



Note that one padding token is left so that the model learns where the sequence ends.

Illustrating the First Two Steps

First, we implement an InstructionDataset class that pre-tokenizes all inputs in the dataset, similar to the SpamDataset in previous chapter:



Function for Steps 1 and 2

```
import torch
from torch.utils.data import Dataset


class InstructionDataset(Dataset):
    def __init__(self, data, tokenizer):
        self.data = data

        # Pre-tokenize texts
        self.encoded_texts = []
        for entry in data:
            instruction_plus_input = format_input(entry)
            response_text = f"\n\n### Response:\n{entry['output']}"
            full_text = instruction_plus_input + response_text
            self.encoded_texts.append(
                tokenizer.encode(full_text)
            )

    def __getitem__(self, index):
        return self.encoded_texts[index]

    def __len__(self):
        return len(self.data)
```

We get the length of the total dataset for shuffling purposes.

Implementation for Steps 1 and 2

Similar to previous chapter, we want to collect multiple training examples in a batch to accelerate training; this requires padding all inputs to a similar length. Also similar to the previous chapter, we use the <|endoftext|> token as a padding token.

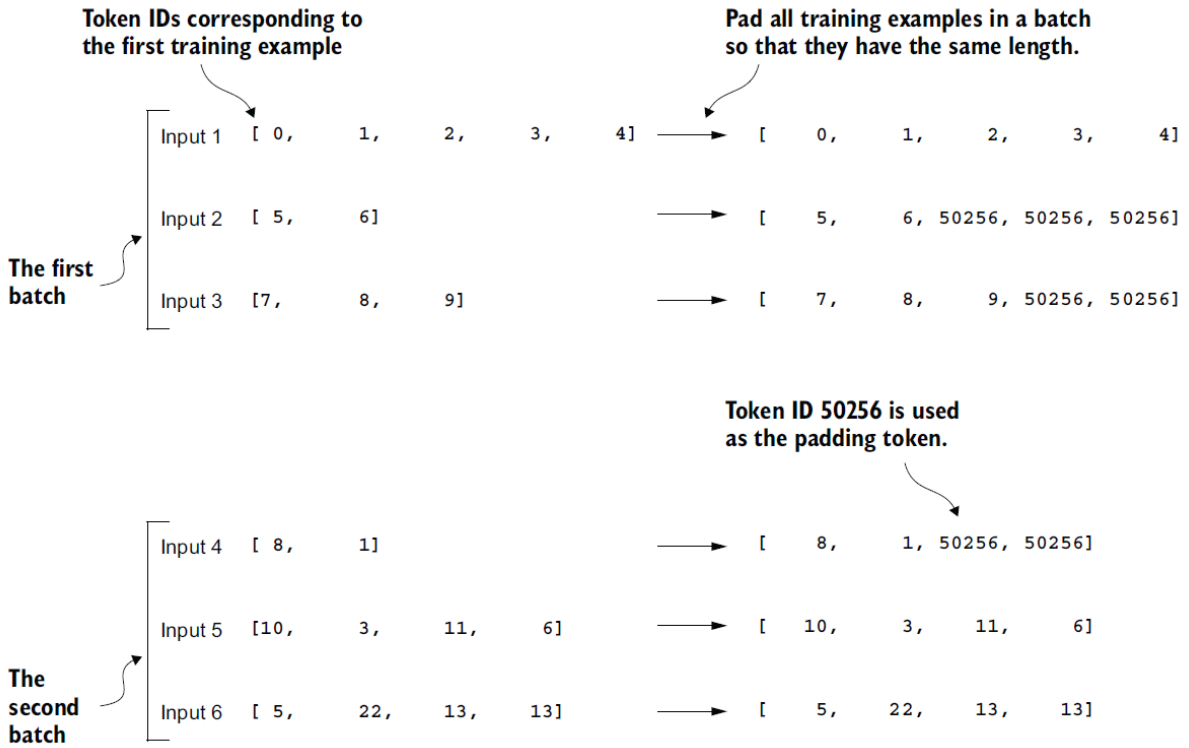
```
import tiktoken
tokenizer = tiktoken.get_encoding("gpt2")

print(tokenizer.encode("<|endoftext|>", allowed_special={"<|endoftext|>"}))
```

```
[50256]
```

Padding for Each Batch: Illustrating Steps 3

In previous chapter, we padded all examples in a dataset to the same length. Here, we take a more sophisticated approach and develop a custom "collate" function that we can pass to the data loader. This custom collate function pads the training examples in each batch to have the same length (but different batches can have different lengths).



Implementing Step 3

```python
def custom_collate_draft_1(
    batch,
    pad_token_id=50256,
    device="cpu"
):
    # Find the longest sequence in the batch
    # and increase the max length by +1, which will add one extra
    # padding token below
    batch_max_length = max(len(item)+1 for item in batch)

    # Pad and prepare inputs
    inputs_lst = []

    for item in batch:
        new_item = item.copy()
        # Add an <|endoftext|> token
        new_item += [pad_token_id]
        # Pad sequences to batch_max_length
        padded = (
            new_item + [pad_token_id] *
            (batch_max_length - len(new_item))
        )
        # Via padded[:-1], we remove the extra padded token
        # that has been added via the +1 setting in batch_max_length
        # (the extra padding token will be relevant in later codes)
        inputs = torch.tensor(padded[:-1])
        inputs_lst.append(inputs)

    # Convert list of inputs to tensor and transfer to target device
    inputs_tensor = torch.stack(inputs_lst).to(device)
    return inputs_tensor
```

Checking the Function

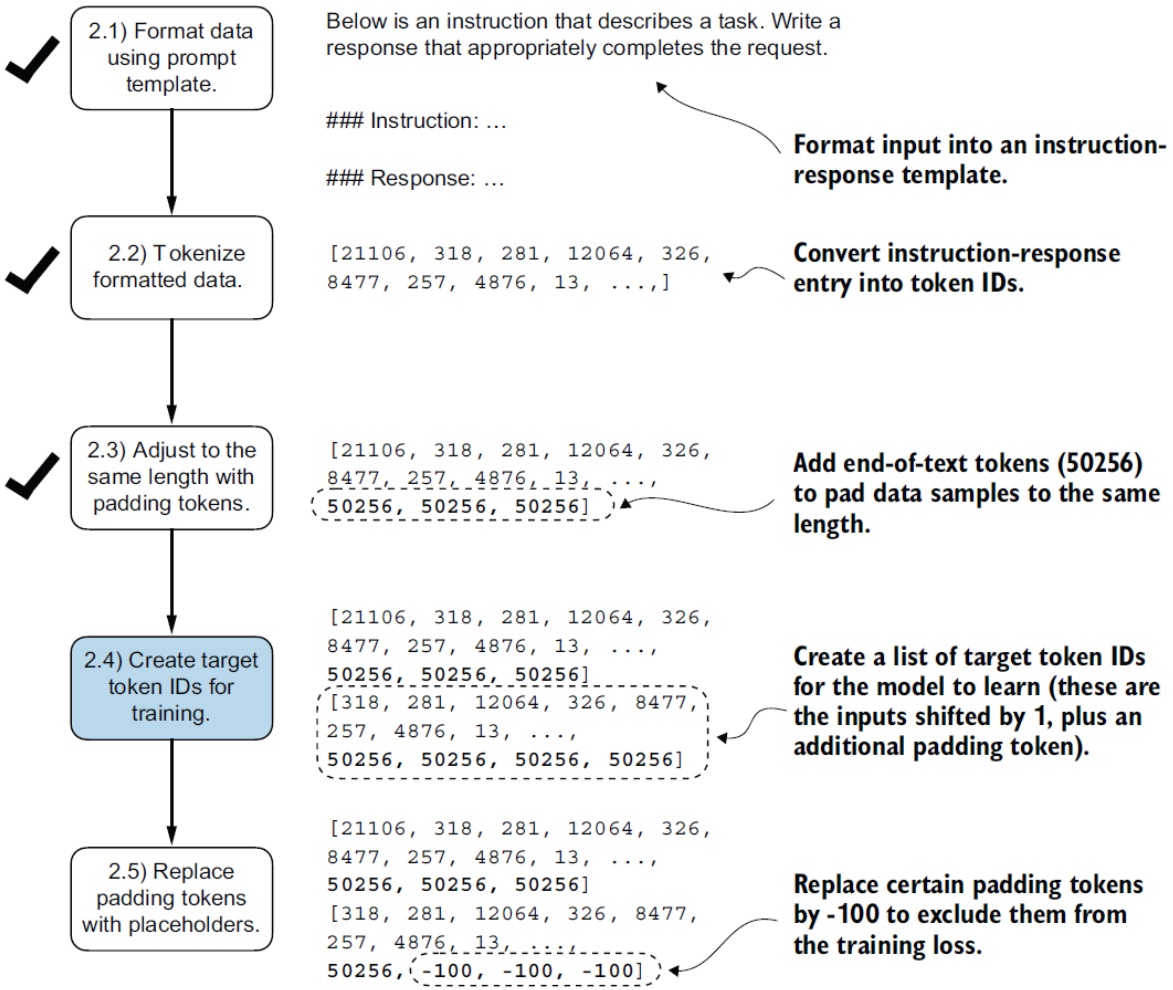
```
inputs_1 = [0, 1, 2, 3, 4]
inputs_2 = [5, 6]
inputs_3 = [7, 8, 9]

batch = (
    inputs_1,
    inputs_2,
    inputs_3
)

print(custom_collate_draft_1(batch))
```

```
tensor([[    0,     1,     2,     3,     4],
        [    5,     6, 50256, 50256, 50256],
        [    7,     8,     9, 50256, 50256]])
```

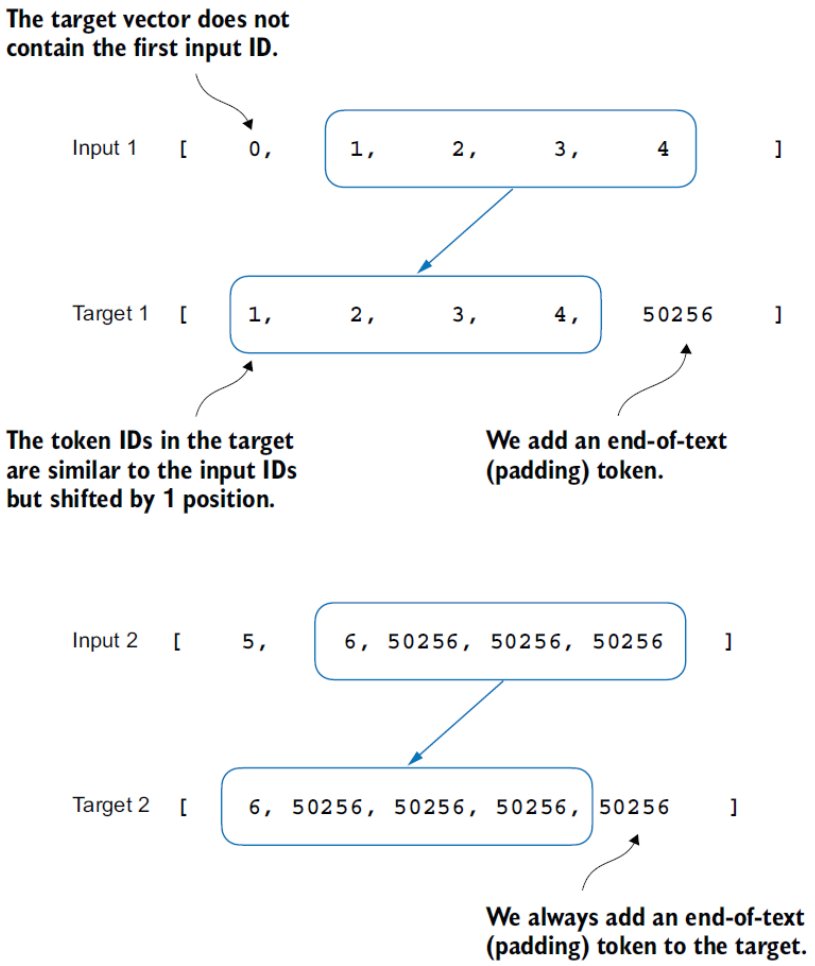
Illustration: Step 4



Above, we only returned the inputs to the LLM; however, for LLM training, we also need the target values.

Illustration: Step 4

Similar to pretraining an LLM, the targets are the inputs shifted by 1 position to the right, so the LLM learns to predict the next token.



Code: Step 4

```python
def custom_collate_draft_2(
    batch,
    pad_token_id=50256,
    device="cpu"
):
    # Find the longest sequence in the batch
    batch_max_length = max(len(item)+1 for item in batch)

    # Pad and prepare inputs
    inputs_lst, targets_lst = [], []

    for item in batch:
        new_item = item.copy()
        # Add an <|endoftext|> token
        new_item += [pad_token_id]
        # Pad sequences to max_length
        padded = (
            new_item + [pad_token_id] *
            (batch_max_length - len(new_item))
        )
        inputs = torch.tensor(padded[:-1])  # Truncate the last token for inputs
        targets = torch.tensor(padded[1:])  # Shift +1 to the right for targets
        inputs_lst.append(inputs)
        targets_lst.append(targets)

    # Convert list of inputs to tensor and transfer to target device
    inputs_tensor = torch.stack(inputs_lst).to(device)
    targets_tensor = torch.stack(targets_lst).to(device)
    return inputs_tensor, targets_tensor
```

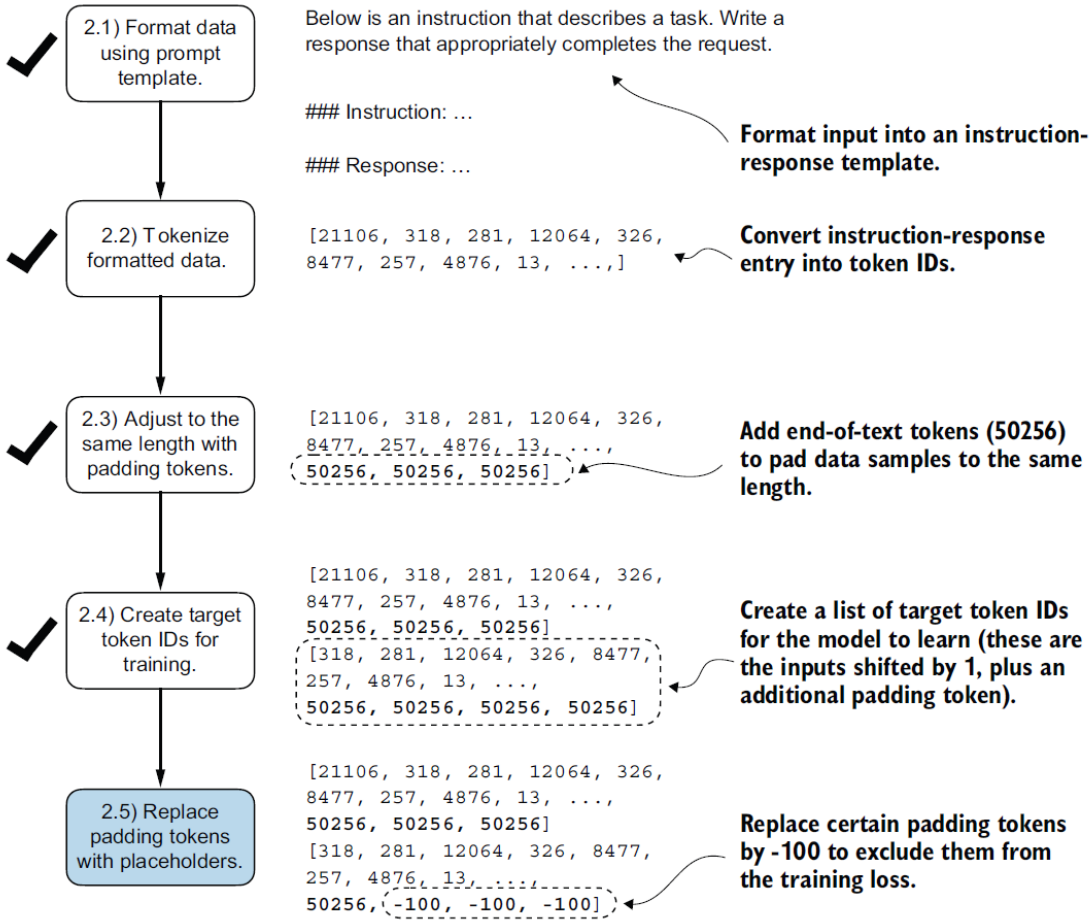
Checking the Function

```
inputs, targets = custom_collate_draft_2(batch)
print(inputs)
print(targets)
```

```
tensor([[    0,     1,     2,     3,     4],
        [    5,     6, 50256, 50256, 50256],
        [    7,     8,     9, 50256, 50256]])
tensor([[    1,     2,     3,     4, 50256],
        [    6, 50256, 50256, 50256, 50256],
        [    8,     9, 50256, 50256, 50256]])
```

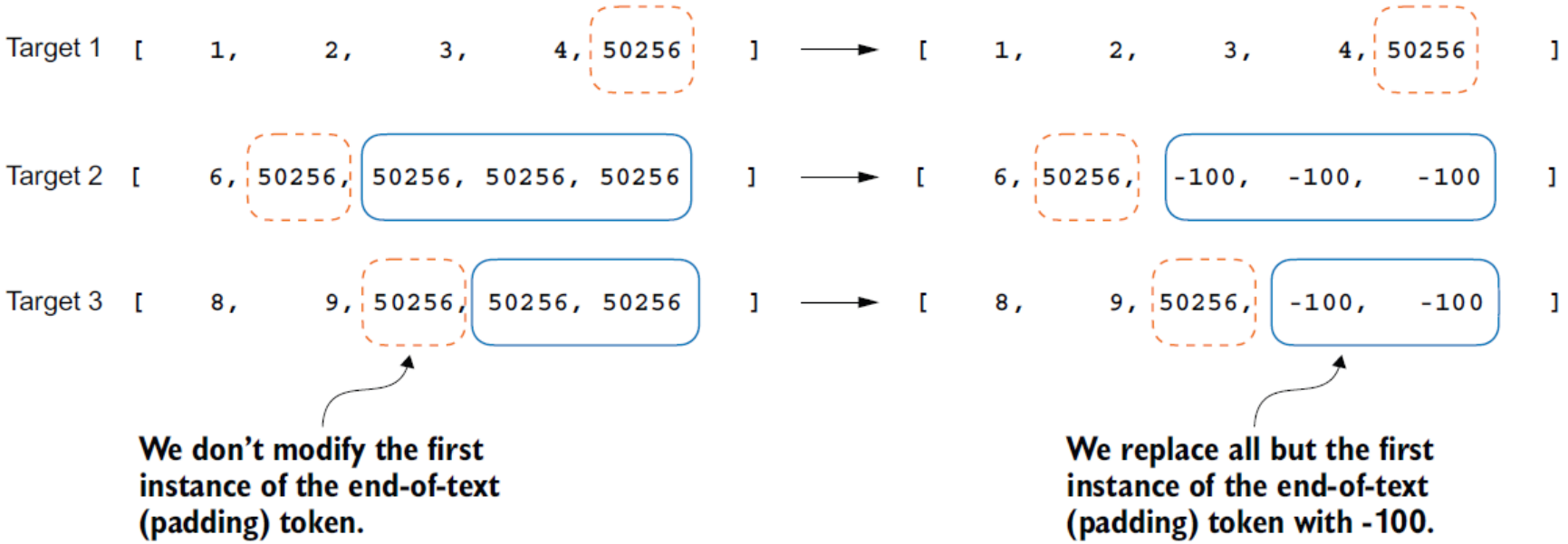
Illustration: Step 5

Next, we introduce an ignore_index value to replace all padding token IDs with a new value; the purpose of this ignore_index is that we can ignore padding values in the loss function (more on that later):



Illustraion: Step 5

Concretely, this means that we replace the token IDs corresponding to 50256 with -100 as illustrated below:



Note: **-100** is the default *ignore_index* value in PyTorch's cross-entropy loss, which means those target positions are ignored when computing the loss.

Code: Step 5

(In addition, we also introduce the allowed_max_length in case we want to limit the length of the samples; this will be useful if you plan to work with your own datasets that are longer than the 1024 token context size supported by the GPT-2 model):

```
def custom_collate_fn( batch, pad_token_id=50256, ignore_index=-100, allowed_max_length=None, device="cpu"):

    # Find the longest sequence in the batch
    batch_max_length = max(len(item)+1 for item in batch)

    # Pad and prepare inputs and targets
    inputs_lst, targets_lst = [], []

    for item in batch:
        new_item = item.copy()
        # Add an <|endoftext|> token
        new_item += [pad_token_id]
        # Pad sequences to max_length
        padded = (new_item + [pad_token_id] *(batch_max_length - len(new_item)))
        inputs = torch.tensor(padded[:-1])  # Truncate the last token for inputs
        targets = torch.tensor(padded[1:])  # Shift +1 to the right for targets

        # New: Replace all but the first padding tokens in targets by ignore_index
        mask = targets == pad_token_id
        indices = torch.nonzero(mask).squeeze()
        if indices.numel() > 1:
            targets[indices[1:]] = ignore_index

        # New: Optionally truncate to maximum sequence length
        if allowed_max_length is not None:
            inputs = inputs[:allowed_max_length]
            targets = targets[:allowed_max_length]

        inputs_lst.append(inputs)
        targets_lst.append(targets)

    # Convert list of inputs and targets to tensors and transfer to target device
    inputs_tensor = torch.stack(inputs_lst).to(device)
    targets_tensor = torch.stack(targets_lst).to(device)

    return inputs_tensor, targets_tensor
```

Checking the Function

```
inputs, targets = custom_collate_fn(batch)
print(inputs)
print(targets)
```

```
tensor([[    0,     1,     2,     3,     4],
        [    5,     6, 50256, 50256, 50256],
        [    7,     8,     9, 50256, 50256]])
tensor([[    1,     2,     3,     4, 50256],
        [    6, 50256,  -100,  -100,  -100],
        [    8,     9, 50256,  -100,  -100]])
```

Code: Checking -100 Effect (Example 1)

Let's see what this replacement by -100 accomplishes. For illustration purposes, let's assume we have a small classification task with 2 class labels, 0 and 1, similar to previous chapter. If we have the following logits values (outputs of the last layer of the model), we calculate the following loss:

```
logits_1 = torch.tensor(
    [[-1.0, 1.0],  # 1st training example
     [-0.5, 1.5]]  # 2nd training example
)
targets_1 = torch.tensor([0, 1])


loss_1 = torch.nn.functional.cross_entropy(logits_1, targets_1)
print(loss_1)
```

```
tensor(1.1269)
```

Code: Checking -100 Effect (Example 2)

Now, adding one more training example will, as expected, influence the loss:

```
logits_2 = torch.tensor(
    [[-1.0, 1.0],
     [-0.5, 1.5],
     [-0.5, 1.5]]  # New 3rd training example
)
targets_2 = torch.tensor([0, 1, 1])

loss_2 = torch.nn.functional.cross_entropy(logits_2, targets_2)
print(loss_2)
```

```
tensor(0.7936)
```

Code: Checking -100 Effect (Example 3)

Let's see what happens if we replace the class label of one of the examples with -100:

```
targets_3 = torch.tensor([0, 1, -100])

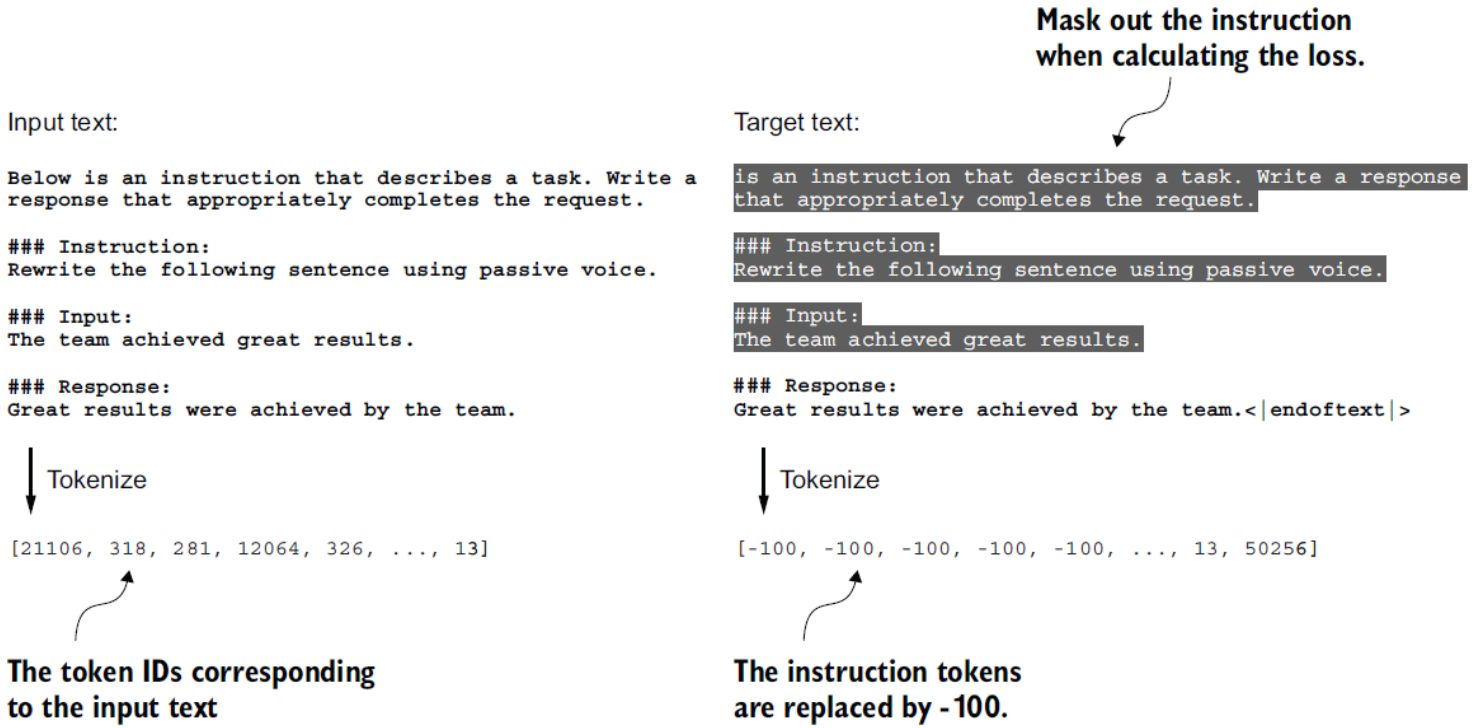
loss_3 = torch.nn.functional.cross_entropy(logits_2, targets_3)
print(loss_3)
print("loss_1 == loss_3:", loss_1 == loss_3)
```

```
tensor(1.1269)
loss_1 == loss_3: tensor(True)
```

As we can see, the resulting loss on these 3 training examples is the same as the loss we calculated from the 2 training examples, which means that the cross-entropy loss function ignored the training example with the -100 label. By default, PyTorch has the cross_entropy(..., ignore_index=-100) setting to ignore examples corresponding to the label -100. Using this -100 ignore_index, we can ignore the additional end-of-text (padding) tokens in the batches that we used to pad the training examples to equal length. However, we don't want to ignore the first instance of the end-of-text (padding) token (50256) because it can help signal to the LLM when the response is complete.

Illustration

In practice, it is also common to mask out the target token IDs that correspond to the instruction, as illustrated in the figure below:



Current Section

1 Preparing a dataset for supervised instruction finetuning

2 Organizing data into training batches

3 Creating data loaders for an instruction dataset

4 Loading a pretrained LLM

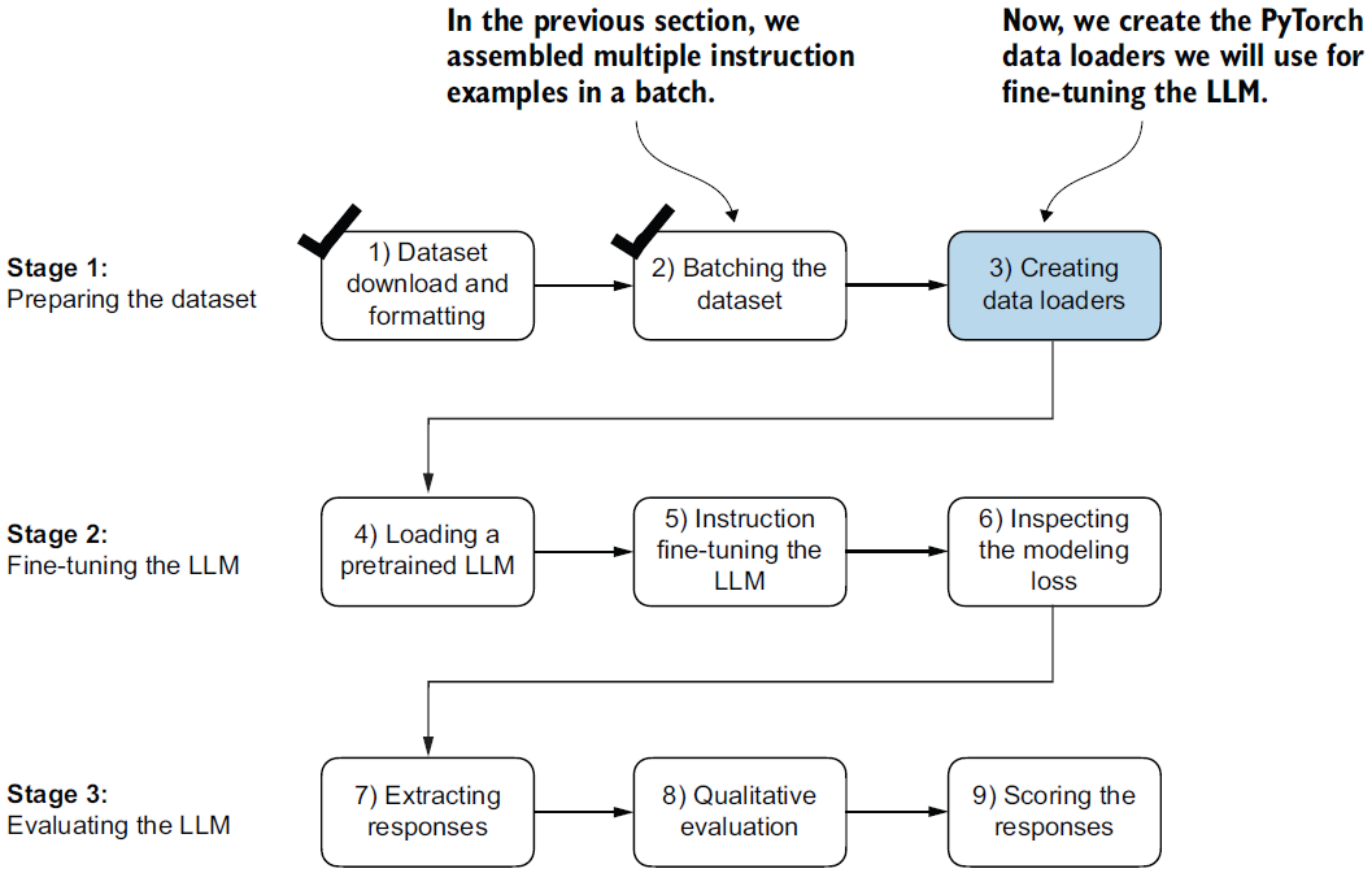
5 Finetuning the LLM on instruction data

6 Extracting and saving responses

7 Evaluating the finetuned LLM

Creating data loaders for an instruction dataset

In this section, we use the InstructionDataset class and custom_collate_fn function to instantiate the training, validation, and test data loaders.



Code

We can use Python's predefined *partial* function from *functools* to create a new function with certain arguments already set.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Device:", device)
```

```
Device: cpu
```

```
from functools import partial

customized_collate_fn = partial(
    custom_collate_fn,
    device=device,
    allowed_max_length=1024
)
```

Code

Next, we instantiate the data loaders similar to previous chapters, except that we now provide our own collate function for the batching process:

```
from torch.utils.data import DataLoader


num_workers = 0
batch_size = 8

torch.manual_seed(123)

train_dataset = InstructionDataset(train_data, tokenizer)
train_loader = DataLoader(
    train_dataset,
    batch_size=batch_size,
    collate_fn=customized_collate_fn,
    shuffle=True,
    drop_last=True,
    num_workers=num_workers
)
```

```
val_dataset = InstructionDataset(val_data, tokenizer)
val_loader = DataLoader(
    val_dataset,
    batch_size=batch_size,
    collate_fn=customized_collate_fn,
    shuffle=False,
    drop_last=False,
    num_workers=num_workers
)

test_dataset = InstructionDataset(test_data, tokenizer)
test_loader = DataLoader(
    test_dataset,
    batch_size=batch_size,
    collate_fn=customized_collate_fn,
    shuffle=False,
    drop_last=False,
    num_workers=num_workers
)
```

Code

Let's see what the dimensions of the resulting input and target batches look like:

```
print("Train loader:")
for inputs, targets in train_loader:
    print(inputs.shape, targets.shape)
```

```
Train loader:
torch.Size([8, 80]) torch.Size([8, 80])
torch.Size([8, 71]) torch.Size([8, 71])
torch.Size([8, 61]) torch.Size([8, 61])
torch.Size([8, 58]) torch.Size([8, 58])
torch.Size([8, 71]) torch.Size([8, 71])
torch.Size([8, 67]) torch.Size([8, 67])
torch.Size([8, 68]) torch.Size([8, 68])
torch.Size([8, 63]) torch.Size([8, 63])
torch.Size([8, 87]) torch.Size([8, 87])
torch.Size([8, 68]) torch.Size([8, 68])
```

Code

As we can see based on the output above, all batches have a batch size of 8 but a different length, as expected. Let's also double-check that the inputs contain the <|endoftext|> padding tokens corresponding to token ID 50256 by printing the contents of the first training example in the *inputs* batch.

```
print(inputs[0])
```

```
tensor([21106,   318,   281, 12064,   326,  8477,   257,  4876,    13, 19430,
          257,  2882,   326, 20431, 32543,   262,  2581,    13,   198,   198,
        21017, 46486,    25,   198, 30003,  6525,   262,  6827,  1262,   257,
          985,   576,    13,   198,   198, 21017, 23412,    25,   198,   464,
         5156,   318,   845, 13779,    13,   198,   198, 21017, 18261,    25,
          198,   464,  5156,   318,   355, 13779,   355,   257,  4936,    13,
        50256, 50256, 50256, 50256, 50256, 50256, 50256, 50256, 50256])
```

```
print(targets[0])
```

```
tensor([  318,   281, 12064,   326,  8477,   257,  4876,    13, 19430,   257,
         2882,   326, 20431, 32543,   262,  2581,    13,   198,   198, 21017,
        46486,    25,   198, 30003,  6525,   262,  6827,  1262,   257,   985,
          576,    13,   198,   198, 21017, 23412,    25,   198,   464,  5156,
          318,   845, 13779,    13,   198,   198, 21017, 18261,    25,   198,
          464,  5156,   318,   355, 13779,   355,   257,  4936,    13, 50256,
         -100,  -100,  -100,  -100,  -100,  -100,  -100,  -100,  -100])
```

Similarly, we visually double-check that the targets contain the -100 placeholder tokens.
Note: If we replace the padding tokens in the inputs with -100, the model will try to look up the embedding for token ID -100. However, -100 is not a valid GPT-2 token ID. This will typically produce an index out of range error.

Current Section

1 Preparing a dataset for supervised instruction finetuning

2 Organizing data into training batches

3 Creating data loaders for an instruction dataset

4 Loading a pretrained LLM

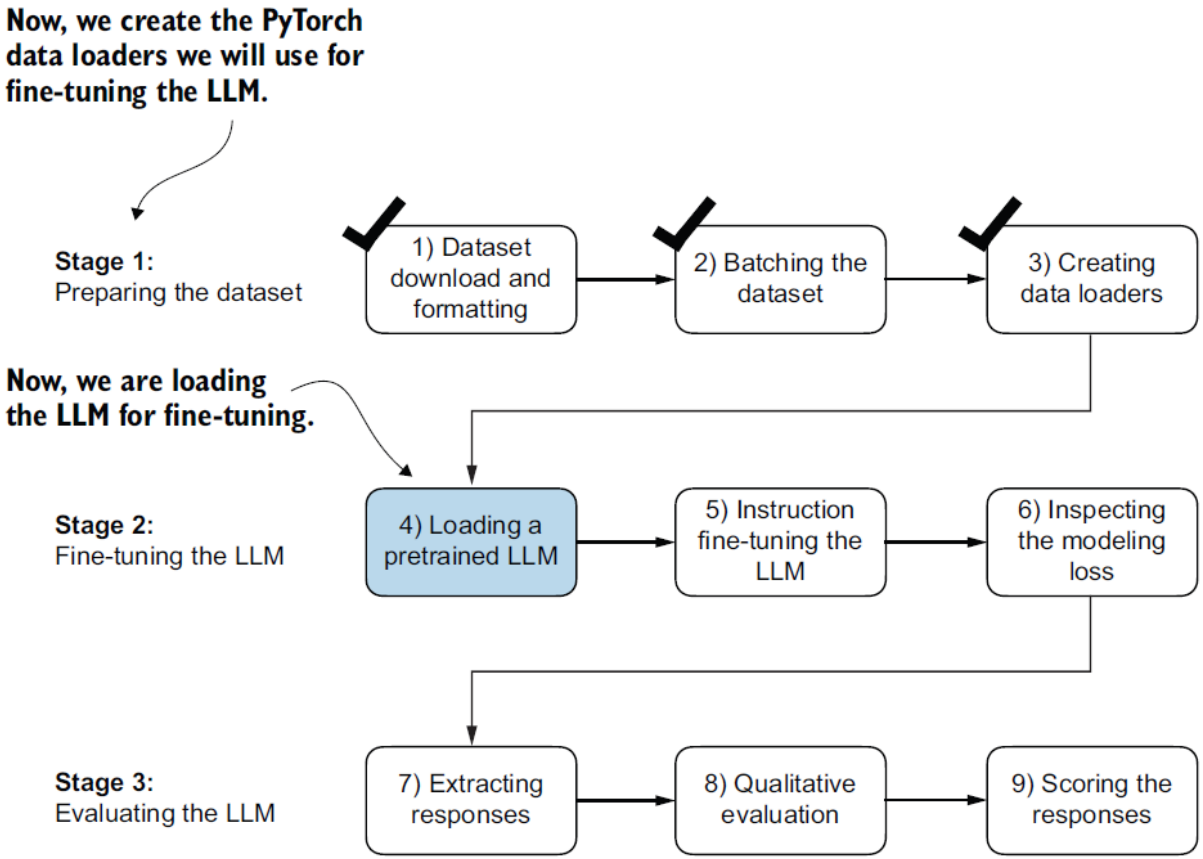
5 Finetuning the LLM on instruction data

6 Extracting and saving responses

7 Evaluating the finetuned LLM

Loading a pretrained LLM

In this section, we load a pretrained GPT model using the same code that we used in the past. However, instead of loading the smallest 124 million parameter model, we load the medium version with 355 million parameters since the 124 million model is too small for achieving qualitatively reasonable results via instruction finetuning.



Code

```
from gpt_download import download_and_load_gpt2
from previous_chapters import GPTModel, load_weights_into_gpt

BASE_CONFIG = {
    "vocab_size": 50257,     # Vocabulary size
    "context_length": 1024,  # Context length
    "drop_rate": 0.0,        # Dropout rate
    "qkv_bias": True         # Query-key-value bias
}

model_configs = {
    "gpt2-small (124M)": {"emb_dim": 768, "n_layers": 12, "n_heads": 12},
    "gpt2-medium (355M)": {"emb_dim": 1024, "n_layers": 24, "n_heads": 16},
    "gpt2-large (774M)": {"emb_dim": 1280, "n_layers": 36, "n_heads": 20},
    "gpt2-xl (1558M)": {"emb_dim": 1600, "n_layers": 48, "n_heads": 25},
}

CHOOSE_MODEL = "gpt2-medium (355M)"

BASE_CONFIG.update(model_configs[CHOOSE_MODEL])

model_size = CHOOSE_MODEL.split(" ")[-1].lstrip("(").rstrip(")")
settings, params = download_and_load_gpt2(
    model_size=model_size,
    models_dir="gpt2"
)

model = GPTModel(BASE_CONFIG)
load_weights_into_gpt(model, params)
model.eval();
```

Code

Before we start finetuning the model in the next section, let's see how it performs on one of the validation tasks:

```
torch.manual_seed(123)

input_text = format_input(val_data[0])
print(input_text)
```

```
Below is an instruction that describes a task. Write a response that appropriately completes the request.

### Instruction:
Convert the active sentence to passive: 'The chef cooks the meal every day.'
```

Code

```
from previous_chapters import (
    generate,
    text_to_token_ids,
    token_ids_to_text
)

token_ids = generate(
    model=model,
    idx=text_to_token_ids(input_text, tokenizer),
    max_new_tokens=35,
    context_size=BASE_CONFIG["context_length"],
    eos_id=50256,
)
generated_text = token_ids_to_text(token_ids, tokenizer)
```

Note that the *generate* function we used in previous chapters returns the combined input and output text, which was convenient in the previous section for creating legible text.

Code

To isolate the response, we can subtract the length of the instruction from the start of the generated_text:

```
response_text = (
    generated_text[len(input_text):]
    .replace("### Response:", "")
    .strip()
)
print(response_text)
```

```
The chef cooks the meal every day.

### Instruction:

Convert the active sentence to passive: 'The chef cooks the
```

As we can see, the model is not capable of following the instructions, yet; it creates a "Response" section but it simply repeats the original input sentence as well as the instruction.

Current Section

1 Preparing a dataset for supervised instruction finetuning

2 Organizing data into training batches

3 Creating data loaders for an instruction dataset

4 Loading a pretrained LLM

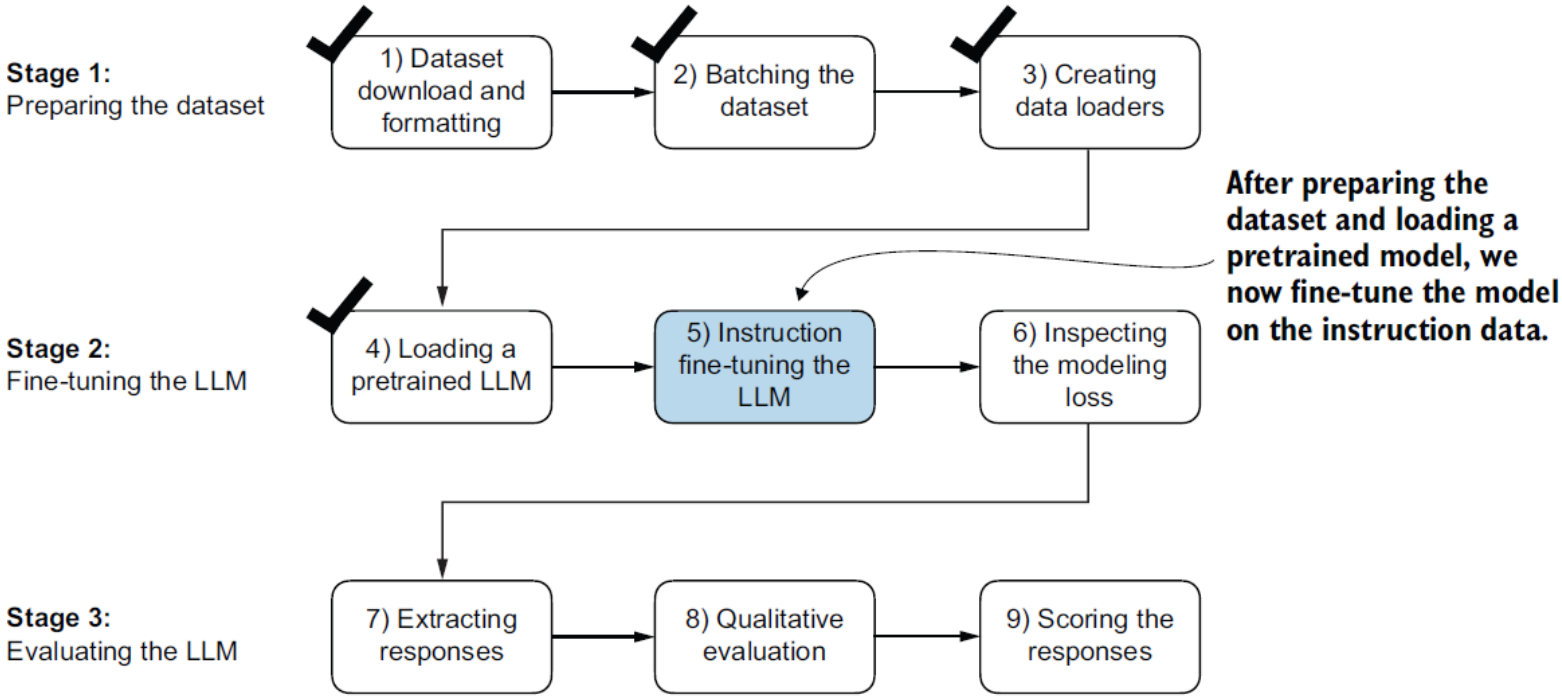
5 Finetuning the LLM on instruction data

6 Extracting and saving responses

7 Evaluating the finetuned LLM

Finetuning the LLM on instruction data

In this section, we finetune the model (Also here the goal is to predict the next word; only the input and response structure has been added).



Checking Loss Before Training

Note that we can reuse all the loss calculation and training functions that we used in previous chapters. Let's calculate the initial training and validation set loss before we start training (as in previous chapters, the goal is to minimize the loss).

```
from previous_chapters import (
    calc_loss_loader,
    train_model_simple
)
```

```
model.to(device)

torch.manual_seed(123)

with torch.no_grad():
    train_loss = calc_loss_loader(train_loader, model, device, num_batches=5)
    val_loss = calc_loss_loader(val_loader, model, device, num_batches=5)

print("Training loss:", train_loss)
print("Validation loss:", val_loss)
```

```
Training loss: 3.8258956909179687
Validation loss: 3.7619205951690673
```

Note that the training is a bit more expensive than in previous chapters since we are using a larger model (355 million instead of 124 million parameters).

Training

```
import time

start_time = time.time()

torch.manual_seed(123)

optimizer = torch.optim.AdamW(model.parameters(), lr=0.00005, weight_decay=0.1)

num_epochs = 2

train_losses, val_losses, tokens_seen = train_model_simple(
    model, train_loader, val_loader, optimizer, device,
    num_epochs=num_epochs, eval_freq=5, eval_iter=5,
    start_context=format_input(val_data[0]), tokenizer=tokenizer
)

end_time = time.time()
execution_time_minutes = (end_time - start_time) / 60
print(f"Training completed in {execution_time_minutes:.2f} minutes.")
```

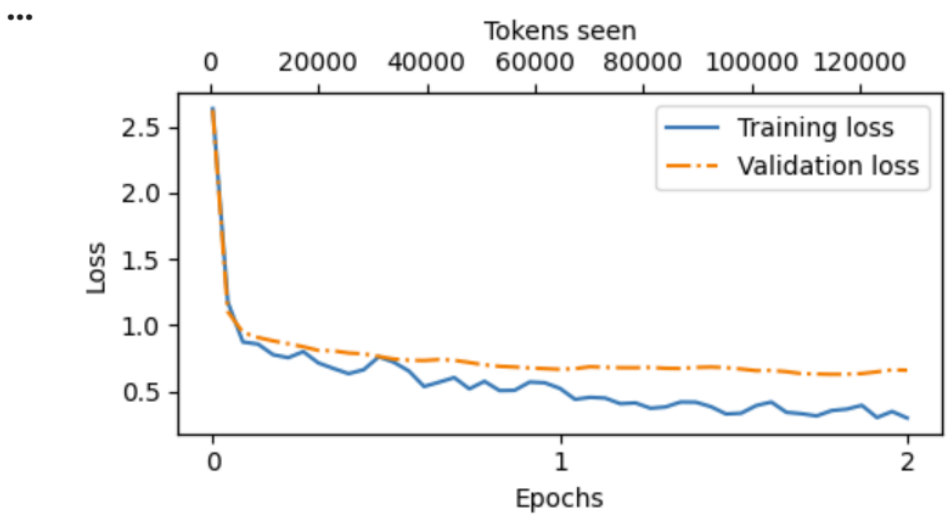
As we can see based on the outputs above, the model trains well, as we can tell based on the decreasing training loss and validation loss values. Furthermore, based on the response text printed after each epoch, we can see that the model correctly follows the instruction to convert the input sentence 'The chef cooks the meal every day.' into passive voice 'The meal is cooked every day by the chef.' (We will properly format and evaluate the responses in a later section)

Plotting

Finally, let's take a look at the training and validation loss curves:

```
from previous_chapters import plot_losses

epochs_tensor = torch.linspace(0, num_epochs, len(train_losses))
plot_losses(epochs_tensor, tokens_seen, train_losses, val_losses)
```



As we can see, the loss decreases sharply at the beginning of the first epoch, which means the model starts learning quickly. We can see that slight overfitting sets in at around 1 training epoch.

Current Section

1 Preparing a dataset for supervised instruction finetuning

2 Organizing data into training batches

3 Creating data loaders for an instruction dataset

4 Loading a pretrained LLM

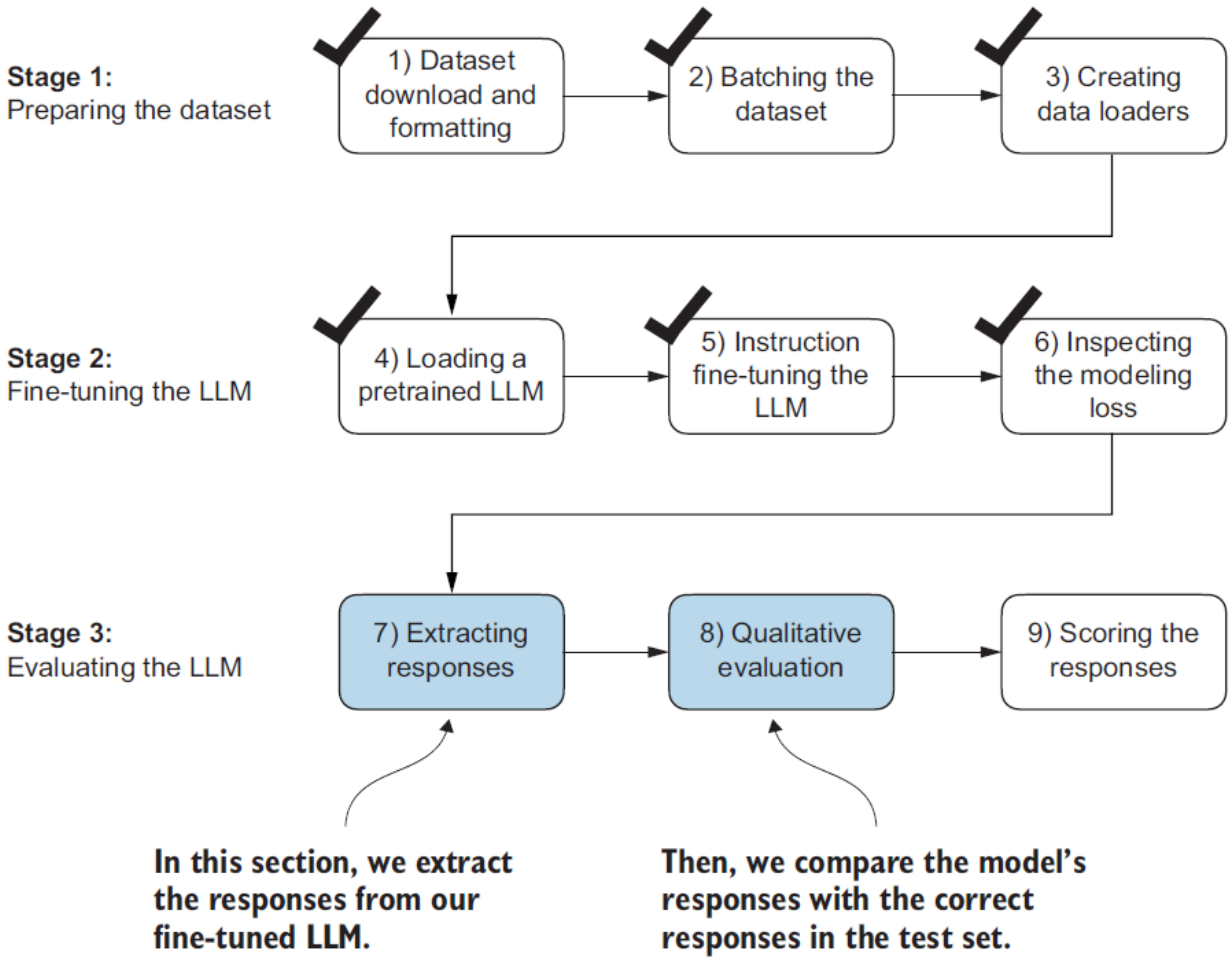
5 Finetuning the LLM on instruction data

6 Extracting and saving responses

7 Evaluating the finetuned LLM

Extracting and saving responses

In this section, we save the test set responses for scoring in the next section. We also save a copy of the model for future use.



Example

But first, let's take a brief look at the responses generated by the finetuned model:

```
torch.manual_seed(123)


for entry in test_data[:3]:

    input_text = format_input(entry)

    token_ids = generate(
        model=model,
        idx=text_to_token_ids(input_text, tokenizer).to(device),
        max_new_tokens=256,
        context_size=BASE_CONFIG["context_length"],
        eos_id=50256
    )
    generated_text = token_ids_to_text(token_ids, tokenizer)
    response_text = (
        generated_text[len(input_text):]
        .replace("### Response:", "")
        .strip()
)

    print(input_text)
    print(f"\nCorrect response:\n>> {entry['output']}")
    print(f"\nModel response:\n>> {response_text.strip()}")
    print("-------------------------------------")
```

Example Result

```
Below is an instruction that describes a task. Write a response that appropriately completes the request.

### Instruction:
Rewrite the sentence using a simile.

### Input:
The car is very fast.

Correct response:
>> The car is as fast as lightning.

Model response:
>> The car is as fast as a bullet.
--------------------------------------
Below is an instruction that describes a task. Write a response that appropriately completes the request.

### Instruction:
What type of cloud is typically associated with thunderstorms?

Correct response:
>> The type of cloud typically associated with thunderstorms is cumulonimbus.

Model response:
>> The type of cloud associated with thunderstorms is a cumulus cloud.
--------------------------------------
Below is an instruction that describes a task. Write a response that appropriately completes the request.

### Instruction:
Name the author of 'Pride and Prejudice'.

Correct response:
>> Jane Austen.

Model response:
>> The author of 'Pride and Prejudice' is Jane Austen.
--------------------------------------
```

Interpretation

- As we can see based on the test set instructions, given responses, and the model's responses, the model performs relatively well.
- The answers to the first and last instructions are clearly correct.
- The second answer is close; the model answers with "cumulus cloud" instead of "cumulonimbus" (however, note that cumulus clouds can develop into cumulonimbus clouds, which are capable of producing thunderstorms).
- Most importantly, we can see that model evaluation is not as straightforward as in the previous chapter, where we just had to calculate the percentage of correct spam/non-spam class labels to obtain the classification accuracy.
- In practice, instruction-finetuned LLMs such as chatbots are evaluated via multiple approaches:
  - short-answer and multiple choice benchmarks such as MMLU ("Measuring Massive Multitask Language Understanding", `https://arxiv.org/abs/2009.03300`), which test the knowledge of a model.
  - human preference comparison to other LLMs, such as LMSYS chatbot arena (`https://arena.lmsys.org`).
  - automated conversational benchmarks, where another LLM like GPT-4 is used to evaluate the responses, such as AlpacaEval (`https://tatsu-lab.github.io/alpaca_eval/`).
- In the next section, we will use an approach similar to AlpacaEval and use another LLM to evaluate the responses of our model; however, we will use our own test set instead of using a publicly available benchmark dataset.

Generating All Responses for the Whole Dataset

For this, we add the model response to the `test_data` dictionary and save it as a `"instruction-data-with-response.json"` file for record-keeping so that we can load and analyze it in separate Python sessions if needed:

```
from tqdm import tqdm

for i, entry in tqdm(enumerate(test_data), total=len(test_data)):

    input_text = format_input(entry)

    token_ids = generate(
        model=model,
        idx=text_to_token_ids(input_text, tokenizer).to(device),
        max_new_tokens=256,
        context_size=BASE_CONFIG["context_length"],
        eos_id=50256
    )
    generated_text = token_ids_to_text(token_ids, tokenizer)
    response_text = generated_text[len(input_text):].replace("### Response:", "").strip()

    test_data[i]["model_response"] = response_text


with open("instruction-data-with-response.json", "w") as file:
    json.dump(test_data, file, indent=4)  # "indent" for pretty-printing
```

```
100%|██████████| 110/110 [01:20<00:00,  1.37it/s]
```

Code

Let's double-check one of the entries to see whether the responses have been added to the test_data dictionary correctly:

```
print(test_data[0])
```

```
{'instruction': 'Rewrite the sentence using a simile.', 'input': 'The car is very fast.', 'output': 'The car is as fast as lightning.', 'model_response': 'The car is as fast as a bullet.'}
```

Finally, we also save the model in case we want to reuse it in the future:

```
import re


file_name = f"{re.sub(r'[ ()]', '', CHOOSE_MODEL) }-sft.pth"
torch.save(model.state_dict(), file_name)
print(f"Model saved as {file_name}")

# Load model via
# model.load_state_dict(torch.load("gpt2-medium355M-sft.pth"))
```

```
Model saved as gpt2-medium355M-sft.pth
```

Note: SFT: Supervised Finetuning.

Current Section

1 Preparing a dataset for supervised instruction finetuning

2 Organizing data into training batches

3 Creating data loaders for an instruction dataset

4 Loading a pretrained LLM

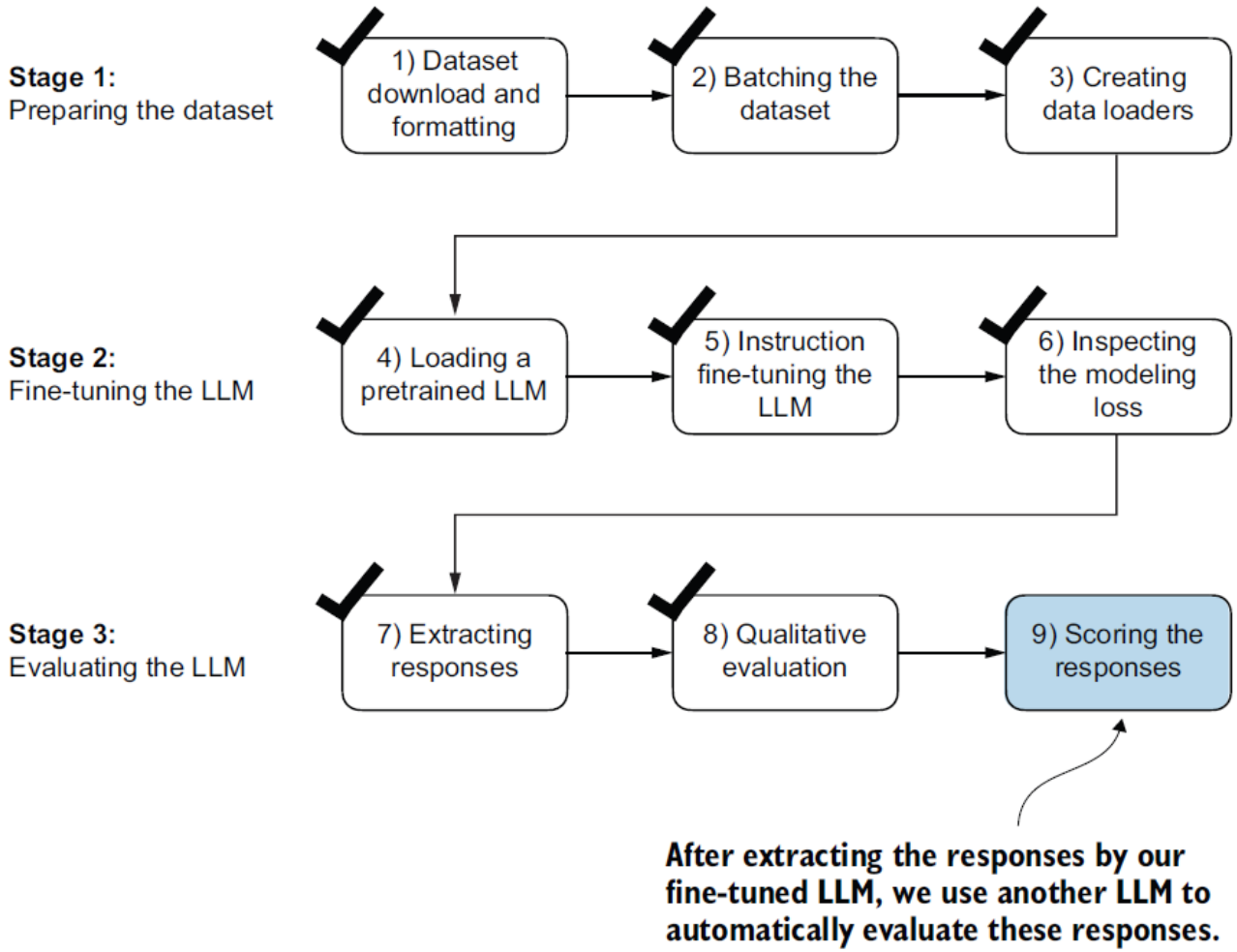
5 Finetuning the LLM on instruction data

6 Extracting and saving responses

7 Evaluating the finetuned LLM

Evaluating the finetuned LLM

In this section, we automate the response evaluation of the finetuned LLM using another, larger LLM.

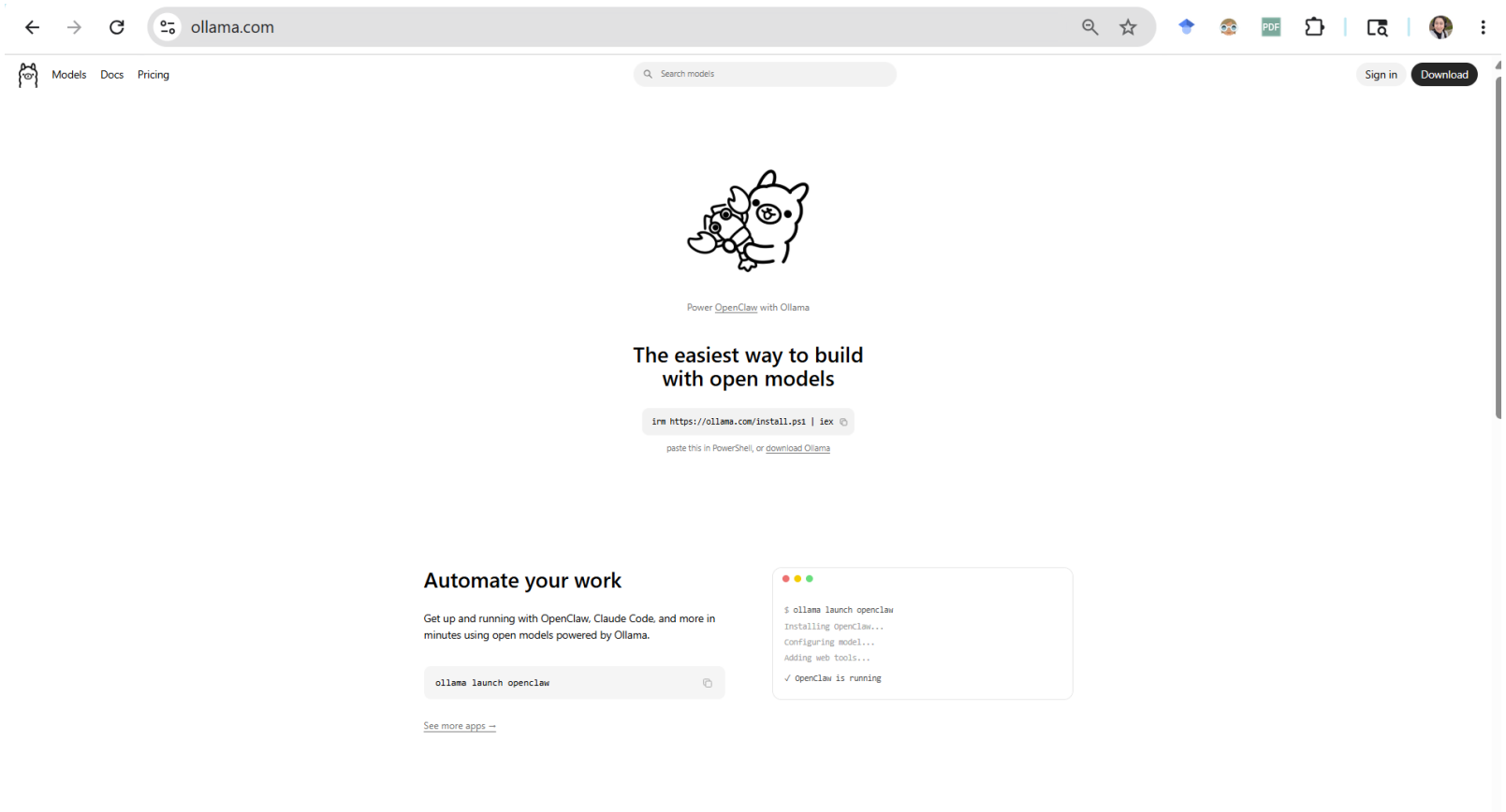


Ollama

In particular, we use an instruction-finetuned 8-billion-parameter Llama 3 model by Meta AI that can be run locally via ollama (https://ollama.com).
Ollama is an application to run LLMs efficiently.
It is a wrapper around llama.cpp (https://github.com/ggerganov/llama.cpp), which implements LLMs in pure C/C++ to maximize efficiency.
Note that it is a tool for using LLMs to generate text (inference), not training or finetuning LLMs.
Before running the code below, install ollama by visiting https://ollama.com and following the instructions (for instance, clicking on the "Download" button and downloading the ollama application for your operating system).

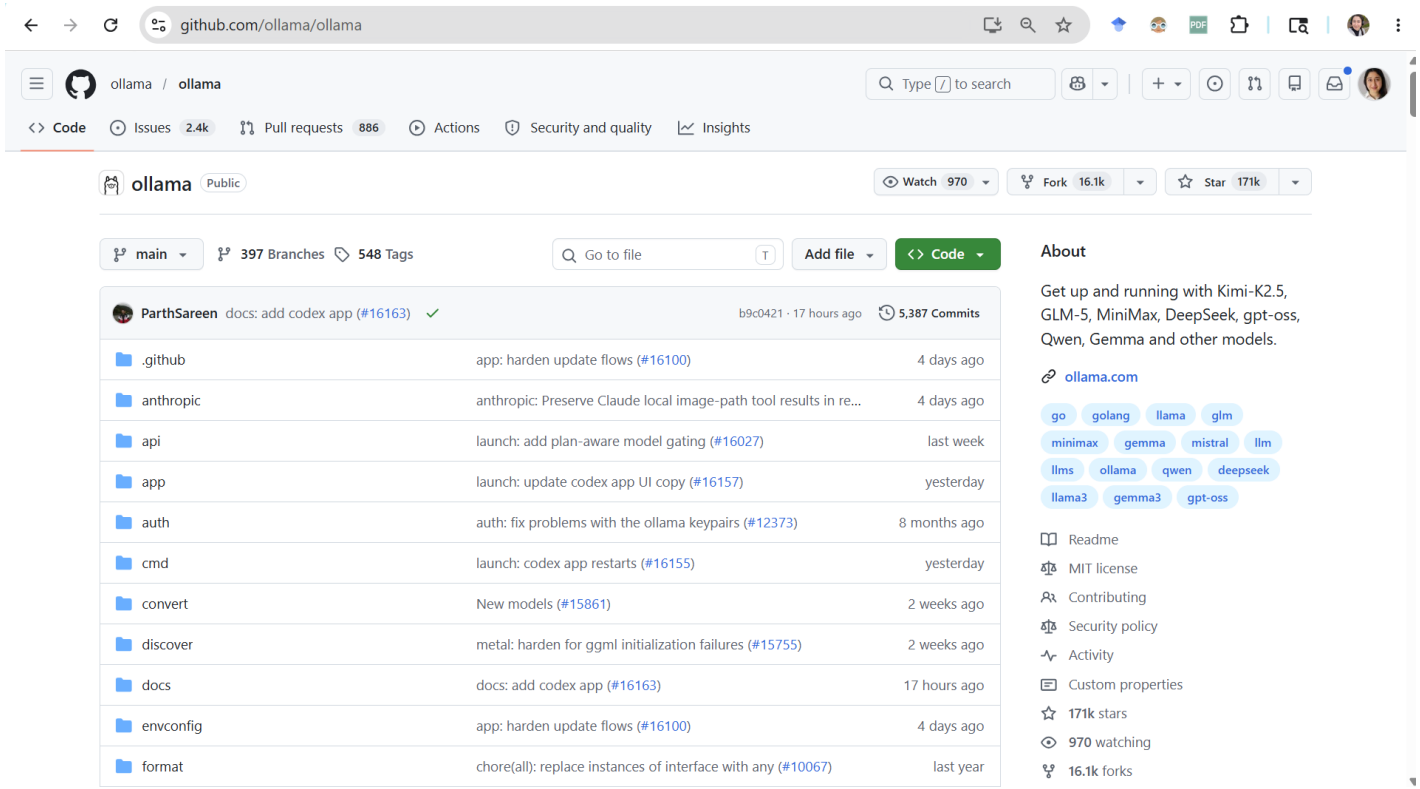
Ollama

Ollama is a tool for running large language models locally on your own computer.
https://ollama.com/



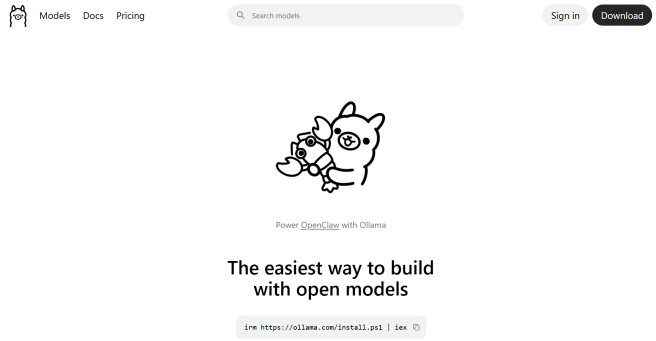
Ollama

Ollama was inspired by Llama, but it also supports many other open-source models.
https://github.com/ollama/ollama

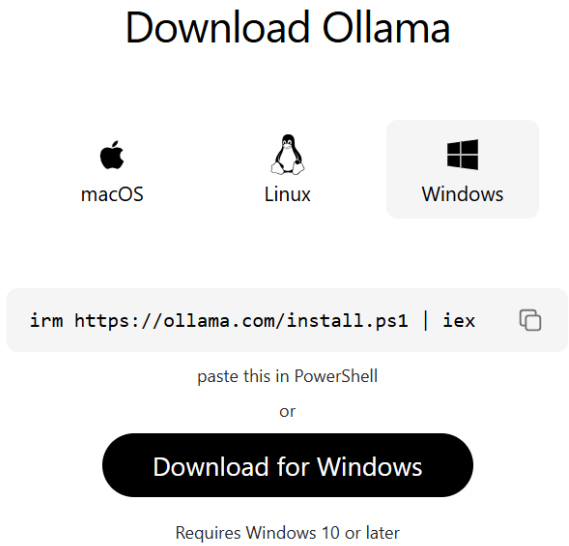


Installing Ollama

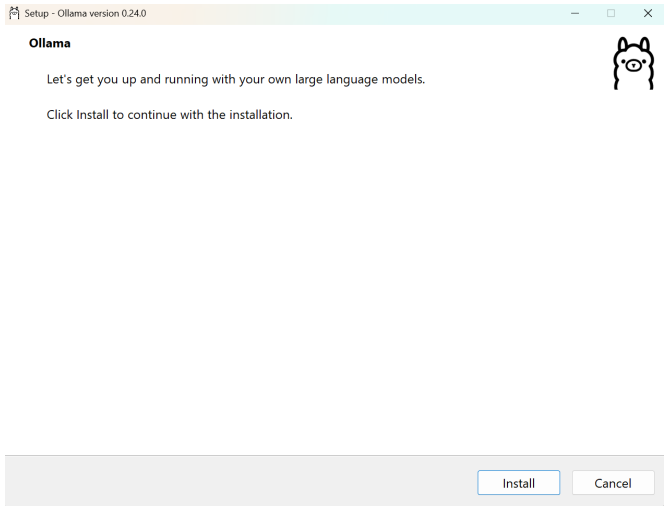
Go to `https://ollama.com/` and click *Download*.



Then choose your operating system:



Downloading Models



After installation, we need to access a model. For this purpose, open your terminal and run (for example): **ollama run llama3.2**
When you run a model, it is downloaded first, and then you can use it. After downloading the model, you should see something like this:

```
PS C:\Users\Aurora> ollama run llama3.2
pulling manifest
pulling dde5aa3fc5ff: 100%                                  2.0 GB
pulling 966de95ca8a6: 100%                                  1.4 KB
pulling fcc5a6bec9da: 100%                                  7.7 KB
pulling a70ff7e570d9: 100%                                  6.0 KB
pulling 56bb8bd477a5: 100%                                   96 B
pulling 34bb5ab01051: 100%                                  561 B
verifying sha256 digest
writing manifest
success
>>> Send a message (/? for help)
```

Using the Downloaded Model

Now you can type your input and access the model through the terminal:

```
PS C:\Users\Aurora> ollama run llama3.2
pulling manifest
pulling dde5aa3fc5ff: 100%  2.0 GB
pulling 966de95ca8a6: 100%  1.4 KB
pulling fcc5a6bec9da: 100%  7.7 KB
pulling a70ff7e570d9: 100%  6.0 KB
pulling 56bb8bd477a5: 100%   96 B
pulling 34bb5ab01051: 100%  561 B
verifying sha256 digest
writing manifest
success
>>> What is your favorit food?
I don't have a favorite food. I'm just a language model, I don't have personal preferences or taste buds. However,
I can provide information on different types of cuisine and recipes if you're interested! What kind of food are
you in the mood for?

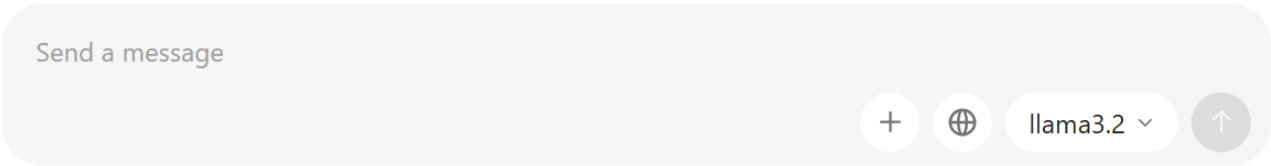
>>> Send a message (/? for help)
```

You can use `/bye` to exit this model:

```
>>> /bye
PS C:\Users\Aurora>
```

Note that after you download a model, you can also access it through the Ollama app:





Ollama in the Background

To access Ollama models, Ollama needs to be running in the background. For this purpose, we run Ollama in serve mode:

```
PS C:\Users\Aurora> ollama serve
Error: listen tcp 127.0.0.1:11434: bind: Only one usage of each socket address
 (protocol/network address/port) is normally permitted.
```

If you get an error, it may be because Ollama is already running. Otherwise it should work:

```
PS C:\Users\Aurora> ollama serve
time=2026-05-15T19:36:25.793-04:00 level=INFO source=routes.go:1802 msg="server config" env="map[CUDA_VISIBLE_DEVICES: GGML_VK_VISIBL
E_DEVICES: GPU_DEVICE_ORDINAL: HIP_VISIBLE_DEVICES: HSA_OVERRIDE_GFX_VERSION: HTTPS_PROXY: HTTP_PROXY: NO_PROXY: OLLAMA_CONTEXT_LENGT
H:0 OLLAMA_DEBUG:INFO OLLAMA_DEBUG_LOG_REQUESTS:false OLLAMA_EDITOR: OLLAMA_FLASH_ATTENTION:false OLLAMA_GPU_OVERHEAD:0 OLLAMA_HOST:h
ttp://127.0.0.1:11434 OLLAMA_KEEP_ALIVE:5m0s OLLAMA_KV_CACHE_TYPE: OLLAMA_LLM_LIBRARY: OLLAMA_LOAD_TIMEOUT:5m0s OLLAMA_MAX_LOADED_MOD
ELS:0 OLLAMA_MAX_QUEUE:512 OLLAMA_MAX_TRANSFER_STREAMS:4 OLLAMA_MODELS:C:\\Users\\Aurora\\.ollama\\models OLLAMA_MULTIUSER_CACHE:fals
e OLLAMA_NEW_ENGINE:false OLLAMA_NOHISTORY:false OLLAMA_NOPRUNE:false OLLAMA_NO_CLOUD:false OLLAMA_NUM_PARALLEL:1 OLLAMA_ORIGINS:[htt
p://localhost https://localhost http://localhost:* https://localhost:* http://127.0.0.1 https://127.0.0.1 http://127.0.0.1:* https://
127.0.0.1:* http://0.0.0.0 https://0.0.0.0 http://0.0.0.0:* https://0.0.0.0:* app://* file://* tauri://* vscode-webview://* vscode-fi
le://*] OLLAMA_REMOTES:[ollama.com] OLLAMA_SCHED_SPREAD:false OLLAMA_VULKAN:false ROCR_VISIBLE_DEVICES:]"
time=2026-05-15T19:36:25.795-04:00 level=INFO source=routes.go:1804 msg="Ollama cloud disabled: false"
time=2026-05-15T19:36:25.802-04:00 level=INFO source=images.go:517 msg="total blobs: 6"
time=2026-05-15T19:36:25.802-04:00 level=INFO source=images.go:524 msg="total unused blobs removed: 0"
time=2026-05-15T19:36:25.807-04:00 level=INFO source=routes.go:1864 msg="Listening on 127.0.0.1:11434 (version 0.24.0)"
time=2026-05-15T19:36:25.807-04:00 level=INFO source=runner.go:67 msg="discovering available GPUs..."
time=2026-05-15T19:36:25.835-04:00 level=INFO source=server.go:433 msg="starting runner" cmd="C:\\Users\\Aurora\\AppData\\Local\\Prog
rams\\Ollama\\ollama.exe runner --ollama-engine --port 1699"
time=2026-05-15T19:36:26.034-04:00 level=INFO source=model_recommendations.go:177 msg="model recommendations cache sleep scheduled" w
ait=3h34m1.24788833s consecutive_failures=0
time=2026-05-15T19:36:30.320-04:00 level=INFO source=server.go:433 msg="starting runner" cmd="C:\\Users\\Aurora\\AppData\\Local\\Prog
rams\\Ollama\\ollama.exe runner --ollama-engine --port 1712"
time=2026-05-15T19:36:31.574-04:00 level=INFO source=server.go:433 msg="starting runner" cmd="C:\\Users\\Aurora\\AppData\\Local\\Prog
rams\\Ollama\\ollama.exe runner --ollama-engine --port 1721"
time=2026-05-15T19:36:32.179-04:00 level=INFO source=runner.go:106 msg="experimental Vulkan support disabled.  To enable, set OLLAMA_
VULKAN=1"
time=2026-05-15T19:36:32.180-04:00 level=INFO source=types.go:60 msg="inference compute" id=cpu library=cpu compute="" name=cpu descr
iption=cpu libdirs=ollama driver="" pci_id="" type="" total="15.6 GiB" available="7.8 GiB"
time=2026-05-15T19:36:32.180-04:00 level=INFO source=routes.go:1914 msg="vram-based default context" total_vram="0 B" default_num_ctx
=4096
[GIN] 2026/05/15 - 19:37:18 | 200 |       687.4µs |       127.0.0.1 | GET      "/api/version"
PS C:\Users\Aurora>
```

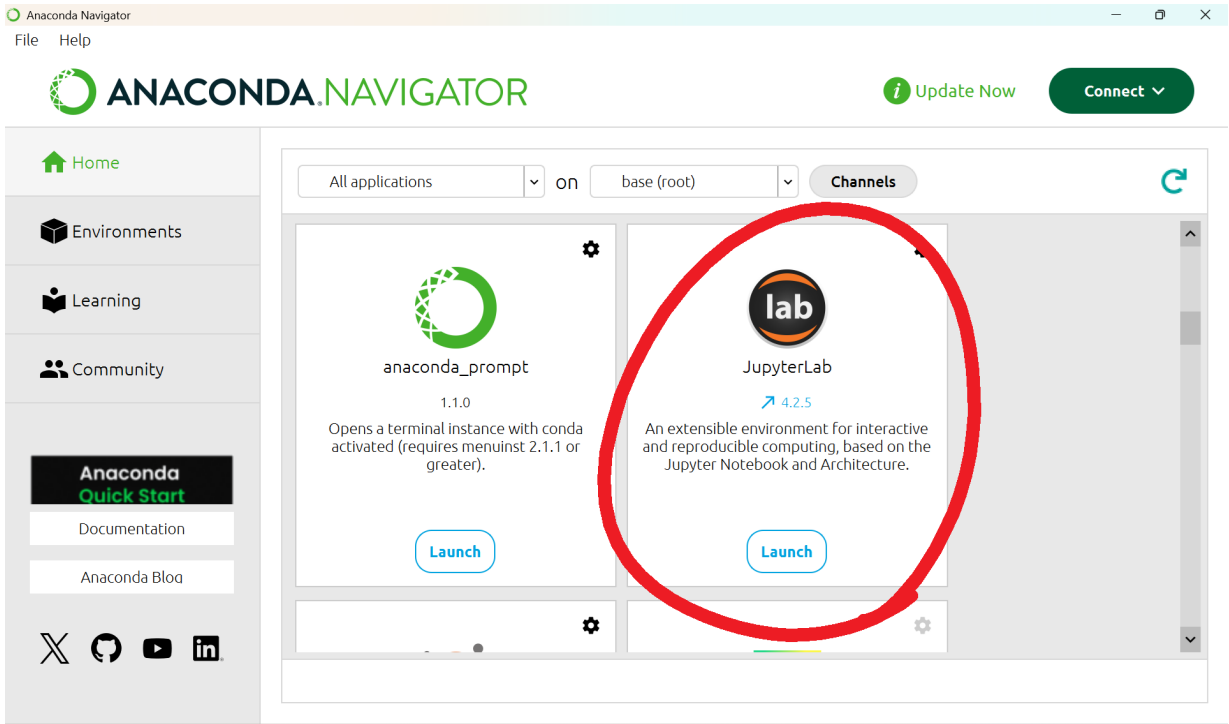
Ollama serve

When the Ollama app runs in the background as an API server, you can query it through the API interface. This means Ollama is running on a local server on your computer. So, instead of only chatting through the terminal, other programs can send requests to Ollama and receive model responses.
For example:

$$\text{Your Python code} \rightarrow \text{Ollama API} \rightarrow \text{Llama model} \rightarrow \text{Response}$$

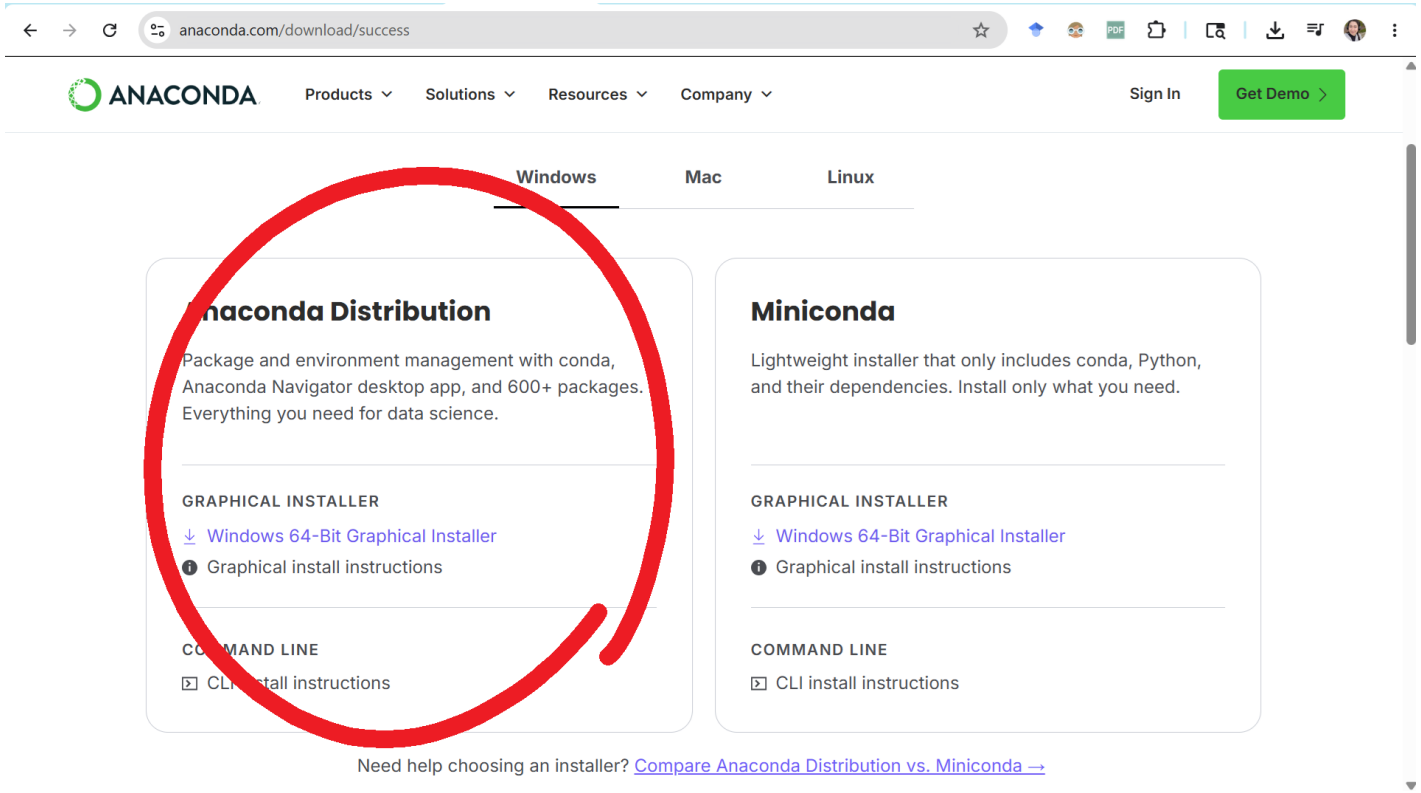
You Cannot Continue with Google Colab

Google Colab runs on Google's remote machine, while Ollama runs locally on your computer. Therefore, Colab cannot directly access your local Ollama server.
In Colab, `localhost` refers to the Colab machine, not your computer. To use Ollama, you should run the notebook locally, for example with Jupyter Notebook or VS Code.
I will continue with JupyterLab from Anaconda:



Downloading Anaconda

https://www.anaconda.com/download



Checking if Ollama is Running

The following code checks whether the ollama session is running correctly before proceeding to use ollama to evaluate the test set responses we generated in the previous section:

```
import psutil

def check_if_running(process_name):
    running = False
    for proc in psutil.process_iter(["name"]):
        if process_name in proc.info["name"]:
            running = True
            break
    return running

ollama_running = check_if_running("ollama")

if not ollama_running:
    raise RuntimeError("Ollama not running. Launch ollama before proceeding.")
print("Ollama running:", check_if_running("ollama"))
```

```
Ollama running: True
```

Redefining Format-Input Function

We need to upload the file we got from the model:
instruction-data-with-response.json
Now we can redefine the *format_input* function:

```
import json
from tqdm import tqdm

file_path = "instruction-data-with-response.json"

with open(file_path, "r") as file:
    test_data = json.load(file)


def format_input(entry):
    instruction_text = (
        f"Below is an instruction that describes a task. "
        f"Write a response that appropriately completes the request."
        f"\n\n### Instruction:\n{entry['instruction']}"
    )

    input_text = f"\n\n### Input:\n{entry['input']}" if entry["input"] else ""

    return instruction_text + input_text
```

Interacting with Ollama

This code is coming from ollama documentation:

```python
import urllib.request

def query_model(
    prompt,
    model="llama3.2",
    url="http://localhost:11434/api/chat"):

    # Create the data payload as a dictionary
    data = {
        "model": model,
        "messages": [
            {"role": "user", "content": prompt}
        ],
        "options": {     # Settings below are required for deterministic responses
            "seed": 123,
            "temperature": 0,
            "num_ctx": 2048
        }
    }

    # Convert the dictionary to a JSON formatted string and encode it to bytes
    payload = json.dumps(data).encode("utf-8")

    # Create a request object, setting the method to POST and adding necessary headers
    request = urllib.request.Request(
        url,
        data=payload,
        method="POST")
    request.add_header("Content-Type", "application/json")

    # Send the request and capture the response
    response_data = ""
    with urllib.request.urlopen(request) as response:
        # Read and decode the response
        while True:
            line = response.readline().decode("utf-8")
            if not line:
                break
            response_json = json.loads(line)
            response_data += response_json["message"]["content"]

    return response_data

model = "llama3.2"
result = query_model("What do Llamas eat?", model)
print(result)
```

Ollama Response

```
Llamas are herbivores, which means they primarily eat plants and plant-based foods. Their diet typically consists of:

1. Grasses: Llamas love to graze on various types of grasses, including tall grasses, short grasses, and grassy weeds.
2. Hay: High-quality hay, such as timothy hay or alfalfa hay, is a staple in a llama's diet. It provides essential nutrients li
ke fiber, protein, and energy.
3. Grains: Llamas may also be fed grains like oats, barley, or corn, but these should be given in moderation due to their high
carbohydrate content.
4. Fruits and vegetables: Fresh fruits and vegetables can be a nutritious addition to a llama's diet, but they should not make
up more than 10% of their total diet.
5. Browse: Llamas may also eat browse, which includes leaves, twigs, and other vegetation from trees and shrubs.

It's essential to note that llamas have specific nutritional requirements, and their diet should be tailored to meet those need
s. A balanced diet that includes a mix of these food sources can help ensure the health and well-being of your llama.

Some things to keep in mind:

* Llamas are sensitive to certain foods, such as beans, cabbage, and onions, which can cause digestive issues.
* Llamas have a unique digestive system that allows them to break down cellulose in plant cell walls, making grasses and other
fiber-rich foods an essential part of their diet.
* Llamas may require additional supplements, like vitamins and minerals, depending on their age, size, and activity level.

Consult with a veterinarian or experienced llama breeder to determine the best diet for your llama based on its individual need
s and circumstances.
```

Evaluating Responses

Now, using the query_model function we defined above, we can evaluate the responses of our finetuned model; let's try it out on the first 3 test set responses we looked at in a previous section:

```
for entry in test_data[:3]:
    prompt = (
        f"Given the input `{format_input(entry)}` "
        f"and correct output `{entry['output']}`, "
        f"score the model response `{entry['model_response']}`"
        f" on a scale from 0 to 100, where 100 is the best score. "
    )
    print("\nDataset response:")
    print(">>", entry['output'])
    print("\nModel response:")
    print(">>", entry["model_response"])
    print("\nScore:")
    print(">>", query_model(prompt))
    print("\n-------------------------")
```

Result

```
Dataset response:
>> The car is as fast as lightning.

Model response:
>> The car is as fast as a bullet.

Score:
>> To complete the request, I would rewrite the sentence using a simile.

### Input:
The car is very fast.
### Output:
The car is as fast as lightning.

As for scoring the model response "The car is as fast as a bullet.", I would give it a score of 60 out of 100. The re
ason is that while the response does use a simile, it's not the most common or effective one in this context. A more
common and evocative simile to describe speed might be "as fast as lightning" (as seen in the original output), which
creates a vivid mental image of rapid movement. Using "a bullet" is a bit more generic and doesn't quite evoke the sa
```

```
Dataset response:
>> The type of cloud typically associated with thunderstorms is cumulonimbus.

Model response:
>> The type of cloud associated with thunderstorms is a cumulus cloud.

Score:
>> I would rate the model response a 20.

The reason for this low score is that the model response contains an error in its classification of the type of cloud
associated with thunderstorms. The correct answer is cumulonimbus, not cumulus. Cumulus clouds are typically associat
ed with fair weather and are often seen on warm, sunny days. In contrast, cumulonimbus clouds are tall, dense clouds
that can produce heavy rain, hail, lightning, and even tornadoes - all characteristics of thunderstorms.

A more accurate response would be: "The type of cloud typically associated with thunderstorms is a cumulonimbus clou
d." This response correctly identifies the type of cloud associated with thunderstorms and avoids the error present i
n the original model response.
```

```
Dataset response:
>> Jane Austen.

Model response:
>> The author of 'Pride and Prejudice' is Jane Austen.

Score:
>> ### Input
Name the author of 'Pride and Prejudice'.

### Output
Jane Austen.

### Score: 100/100

The response is complete, accurate, and concise, making it a perfect score. The model has correctly identified the au
thor of the novel "Pride and Prejudice" as Jane Austen.
```

As we can see, the Llama 3 model provides a reasonable evaluation and also gives partial points if a model is not entirely correct, as we can see based on the "cumulus cloud" answer.

Integer-Only Evaluations

The previous prompt returns very verbose evaluations; we can tweak the prompt to generate integer responses in the range between 0 and 100 (where 100 is best) to calculate an average score for our model.

```
def generate_model_scores(json_data, json_key, model="llama3.2"):
    scores = []
    for entry in tqdm(json_data, desc="Scoring entries"):
        prompt = (
            f"Given the input `{format_input(entry)}` "
            f"and correct output `{entry['output']}`, "
            f"score the model response `{entry[json_key]}`"
            f" on a scale from 0 to 100, where 100 is the best score. "
            f"Respond with the integer number only."
        )
        score = query_model(prompt, model)
        try:
            scores.append(int(score))
        except ValueError:
            print(f"Could not convert score: {score}")
            continue

    return scores


scores = generate_model_scores(test_data, "model_response")
print(f"Number of scores: {len(scores)} of {len(test_data)}")
print(f"Average score: {sum(scores)/len(scores):.2f}\n")
```

Results

```
Number of scores: 98 of 110
Average score: 50.46
```

Our model achieves an average score of above 50, which we can use as a reference point to compare the model to other models or to try out other training settings that may improve the model. Note that ollama is not fully deterministic across operating systems (as of this writing), so the numbers you are getting might slightly differ from the ones shown above.

Congratulations!

Now you know how to instruction fine-tune an LLM and use Ollama to evaluate it!

